Norwegian
University of
Life Sciences

Master's Thesis 2026 **30 ECTS**
Faculty of Science and Technology

Efficient Low-Rank Methods for PDE-Based Inverse Problems

Elias Jegervatn Severinsen
Data Science

Abstract

We study source identification as an elliptic PDE-constrained inverse problem, where the forward operator has rapid decay of singular values and a large nullspace, making solutions unstable and non-unique. The use of Tikhonov regularization results in biased solutions, concentrating reconstructions near the boundary of the domain regardless of the true source location. True sources are often sparse, yet Tikhonov yields smooth reconstructions, blending distinct sources into connected regions.

Elvetun and Nielsen correct the bias by applying a regularization operator, but the procedure requires explicit assembly of the forward operator and its singular value decomposition (SVD), making it infeasible at scale. It also suffers from the same smoothing issues as the Tikhonov method, making low-rank solutions a more natural approach for sparse sources. Dynamical low-rank approximation (DLRA) provides a robust framework for computing them but converges slowly for ill-posed problems.

We propose a matrix-free variant of the randomized SVD (rSVD), avoiding explicit construction of the forward operator, and making the weighted Tikhonov approach scalable. We also apply conjugate gradient (CG) within the DLRA framework (DLRA-CG) to accelerate convergence and recover sparse sources.

We apply the matrix-free rSVD approach to previously inaccessible problems with no significant loss in reconstruction quality. Using DLRA-CG, we recover low-rank solutions that separate distinct sources more clearly and achieve lower reconstruction error than full-rank weighted Tikhonov. The method converges substantially faster than DLRA.

Together, these contributions make large-scale sparse source identification possible, addressing the computational cost of the weighted Tikhonov approach and the tendency of these methods to blur distinct sources.

Sammendrag

Vi studerer kildeidentifikasjon som et elliptisk PDE-betinget inverst problem, der foroveroperatoren har raskt avtagende singulærverdier og et stort nullrom, noe som gjør løsninger ustabile og ikke-entydige. Bruk av Tikhonov-regularisering gir forventningsskjeve løsninger som konsentrerer seg nær randen av domenet, uavhengig av den sanne kildeposisjonen. Sanne kilder er ofte lokaliserte, men Tikhonov gir glatte løsninger hvor distinkte kilder blir slått sammen til sammenhengende områder.

Elvetun og Nielsen retter opp skjevheten ved å anvende en regulariseringsoperator, men prosedyren krever eksplisitt konstruksjon av foroveroperatoren og dens singulærverdidekomposisjon (SVD), noe som gjør den uegnet for store problemer. Den lider også av de samme utglattingsproblemene som Tikhonov-metoden, noe som gjør lavrangsløsninger til en mer naturlig tilnærming for lokaliserte kilder. Dynamisk lavrangs-approksimasjon (DLRA) gir et robust rammeverk for å finne slike kilder, men konvergerer sakte for dårlig stilte problemer.

Vi foreslår en matrisefri variant av den randomiserte SVD-en (rSVD) som unngår eksplisitt konstruksjon av foroveroperatoren og gjør den vektede Tikhonov-tilnærmingen skalerbar. Vi anvender også konjugert gradient (CG) innenfor DLRA-rammeverket (DLRA-CG) for å akselerere konvergensen og gjenvinne separerte kilder.

Den matrisefrie rSVD-tilnærmingen gjør det mulig å løse problemer som tidligere var utenfor rekkevidde, uten betydelig tap i rekonstruksjonskvalitet. DLRA-CG gir lavrangs-løsninger som skiller distinkte kilder tydeligere og oppnår lavere rekonstruksjonsfeil. Metoden konvergerer vesentlig raskere enn DLRA.

Samlet sett gjør disse bidragene storskala identifikasjon av distinkte kilder mulig, ved å redusere den beregningsmessige kostnaden til den vektede Tikhonov-tilnærmingen og motvirke tendensen metodene har til å slå sammen distinkte kilder.

Acknowledgements

I thank my supervisor Ole L. Elvetun and co-supervisor Jonas Kusch for their ideas, discussions, careful proofreading, and feedback throughout this thesis.

I also thank my mom and dad for their continuous support throughout my studies.

Contents

List of figures

List of tables

List of symbols

Geometry

Ω	Computational domain	3
$\partial\Omega$	Boundary of the domain	3
$\mathcal{M}$	Triangulation (mesh)	4
N	Number of mesh nodes	4
N_b	Number of boundary mesh nodes	4

PDE notation

σ	Diffusion coefficient	3
c	Reaction coefficient	3
f	Source function	3

Continuous operators

$\mathcal{S}$	PDE operator	3
$\mathcal{T}$	Trace operator	3
$\mathcal{K}$	Forward operator	3
$\mathcal{K}^*$	Adjoint operator	6

Function spaces

$L^2(\Omega)$	Space of square integrable functions	3
$H^1(\Omega)$	First-order Sobolev space	3
V_h	Finite element space	4
ϕ_i	Finite element basis function	4

Discrete operators & Tikhonov

A_h	Stiffness matrix	5
M	Volume mass matrix	5
K	Discrete forward operator	5
M_∂	Boundary mass matrix	6
K^*	Discrete adjoint operator	6
W	Tikhonov weight matrix	7
λ	Regularization parameter	7

Singular value decomposition

U	Left singular vector matrix	7
V	Right singular vector matrix	7
Σ	Singular value matrix	7

Randomized SVD

Dynamical low-rank approximation

Note on the use of AI

Generative AI (Claude by Anthropic) was used for proofreading and coding assistance. All generated content was reviewed by the author.

1 Introduction

Elliptic partial differential equations (PDEs) model steady-state problems such as electrical potential, heat conduction, and diffusion processes [1], and have applications in engineering, finance, and environmental science, among others [2,3].

We study source identification constrained by an elliptic PDE: given a domain Ω with boundary $\partial\Omega$ and boundary observations $y = u|_{\partial\Omega}$ of the solution $u = \mathcal{S}f$, where $\mathcal{S}$ is the solution operator of the PDE, we seek to recover the unknown interior source f. This leads to an inverse problem of the form

$$\min_f \|u - y\|^2_{L^2(\partial\Omega)} + \lambda^2 \|Wf\|^2_{L^2(\Omega)} \tag{1}$$

subject to

$$\begin{aligned} -\nabla \cdot \sigma \nabla u + cu &= f \quad \text{in } \Omega, \\ \frac{\partial u}{\partial n} &= 0 \quad \text{on } \partial\Omega. \end{aligned} \tag{2}$$

Here, λ is a regularization parameter, W is the regularization weight operator, σ is the diffusion coefficient and c is the reaction coefficient. We refer to the operator $\mathcal{K} : f \mapsto u|_{\partial\Omega}$ as the forward operator, mapping the source to the boundary values of the solution. The Neumann boundary condition in Equation (2) ensures no flow over the boundary.

This problem arises in applications such as inverse imaging, crack determination, and electromagnetic brain mapping [4,5,6]. For example, in electroencephalography (EEG), it can be used for improved understanding and treatment of brain disorders like Alzheimer's, Parkinson's, and depression [5,7].

However, source identification problems of this form are ill-posed, as the operator $\mathcal{K}$ has a rapid decay of singular values and a large nullspace. Reconstructions are therefore unstable and non-unique [8]. Standard approaches like Tikhonov regularization introduce bias to the reconstructions, making solutions appear near the domain boundary regardless of the true location of the source [9]. True sources are often sparse, yet Tikhonov smooths reconstructions, blurring distinct sources into connected regions.

Elvetun and Nielsen address the localization bias of Tikhonov by introducing regularization weights W that use information about the forward operator $\mathcal{K}$ to correct the bias [9]. This method shows promising results, correctly identifying the position of one or multiple sources from boundary data [9,10]. However, their approach requires explicit knowledge about $\mathcal{K}$ and its SVD, making it computationally infeasible for large-scale problems. Also, for sparse sources, Elvetun and Nielsen's method still suffers from the same smoothing problems as Tikhonov, blurring the reconstructions and blending distinct sources [10]. Since sparse sources are naturally represented by low-rank matrices, recovering low-rank solutions is an appropriate objective. Dynamical low-rank approximation (DLRA) provides a robust framework for this [11], but existing methods rely on first-order updates such as Adam [12] and steepest descent, which converge slowly for ill-conditioned problems [13,8,14].

We propose two methods. The first is a matrix-free variant of the randomized SVD (rSVD) that computes a rank-k factorization of the forward operator, avoiding its explicit formation and addressing the computational cost of Elvetun and Nielsen's weights. The second recovers sparse, rank-r sources where Tikhonov smooths, converging substantially faster than existing DLRA methods. The ranks k and r are independent: k is the approximation rank of $\mathcal{K}$, and r is the rank of the matrix representation of the source f.

The first method uses random sampling of range(K) to compute an approximate rank-k SVD $K \approx U\Sigma V^T$ of the discretized forward operator, which can then be used to solve the weighted Tikhonov problem at large scale. It relies on applications of the forward operator K and adjoint K^*, which can be done efficiently by pre-computing sparse Cholesky factorizations of the PDE operator. The method is applied to previously computationally infeasible problems without a significant loss in reconstruction quality.

The second method uses conjugate gradient (CG) within the DLRA framework to accelerate convergence and recover sparse sources. It adapts the DLRA method proposed by Schotthöfer et al. for neural network training [11] to the setting of sparse source identification. The convex optimization problem of Equation (1), with Elvetun and Nielsen's W, defines an objective function Φ with a symmetric positive definite (SPD) Hessian H, for which CG is particularly well-suited [15,16]. The resulting method converges substantially faster than DLRA with steepest descent and yields rank-r solutions that separate distinct sources more clearly than full-rank weighted Tikhonov.

The proposed methods are not limited to the particular problem studied in this thesis. The matrix-free rSVD method is well suited for a broader class of elliptic source identification problems, where the forward operators have rapid spectral decay [2,17]. The DLRA-CG method can be used for low-rank problems with a quadratic objective function and an SPD Hessian.

The remainder of this thesis is organized as follows. Section 2 covers preliminaries and describes the discretization of Equation (1) and the weights of Elvetun and Nielsen. Section 3 presents the matrix-free rSVD (Algorithm 2) and a variant of it applied to K^* (Algorithm 3). Section 4 motivates DLRA and introduces the DLRA-CG method (Algorithm 5) and a preconditioned variant (Algorithm 6). Section 5 reports numerical experiments for both methods, examining computational cost, reconstruction error, and convergence rates. Section 6 discusses the results, and Section 7 concludes with limitations and future directions.

2 Mathematical background

2.1 The forward operator

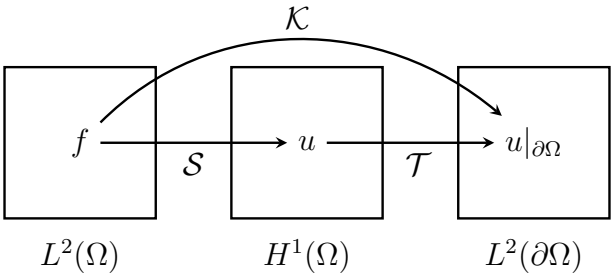


Figure 1: The forward operator $\mathcal{K} = \mathcal{T} \circ \mathcal{S}$ maps f to $u|_{\partial\Omega}$.

Let $\Omega \subset \mathbb{R}^2$ be a bounded domain with boundary $\partial\Omega$, and consider the boundary value problem (BVP) in Equation (2). The coefficient $\sigma = \sigma(x) > 0$ denotes the diffusion coefficient and determines the rate and direction of diffusion in the problem. It may be either scalar-valued or matrix-valued. In the latter case, we assume that $\sigma(x)$ is symmetric and positive definite (see Appendix A.3) to keep the problem elliptic. The reaction coefficient $c = c(x) > 0$ describes the absorption or decay within the problem. Finally, $f = f(x)$ denotes the source term, which is the unknown input of the inverse problem.

Equation (2) is a linear diffusion-reaction problem with Neumann boundary conditions, and defines a mapping

$$\mathcal{S} : L^2(\Omega) \to H^1(\Omega), \quad \mathcal{S}f = u. \tag{3}$$

Here, $L^2(\Omega)$ is the space of square integrable functions and $H^1(\Omega) \subset L^2(\Omega)$ a first-order Sobolev space, see Appendix A.1.

To model boundary observations, we introduce the trace operator

$$\mathcal{T} : H^1(\Omega) \to L^2(\partial\Omega), \quad \mathcal{T}u = u|_{\partial\Omega}. \tag{4}$$

The term $\mathcal{T}u$ represents the boundary observations of the solution u.

The forward operator is then defined as the composition

$$\mathcal{K} = \mathcal{T} \circ \mathcal{S}, \quad \mathcal{K}f = u|_{\partial\Omega}, \tag{5}$$

which is a mapping from the source function f to the observed output $u|_{\partial\Omega} = \mathcal{K}f$ on the boundary $\partial\Omega$, see Figure 1.

The associated inverse problem can now be formulated as follows: given boundary data $u|_{\partial\Omega}$, determine a source function f such that

$$\mathcal{K}f = u|_{\partial\Omega}.$$

Solving this problem computationally requires discretizing $\mathcal{K}$, understanding the ill-posedness of the resulting system, and regularizing it to obtain stable solutions, which we address in the following sections.

2.1.1 Ill-posedness

A problem is said to be well-posed if its solution exists, is unique, and is stable in the sense that it depends continuously on the data, as formulated by Hadamard [8]. Denoting $y = u|_{\partial\Omega}$, the inverse problem $\mathcal{K}f = y$ is ill-posed.

The null space of $\mathcal{K}$ is non-trivial: any f_0 satisfying $\mathcal{K}f_0 = 0$ can be added to a solution without changing the boundary data y. The solution is therefore not unique.

The instability of the solution f is more severe in practice. The compactness of $\mathcal{K}$, and the ellipticity of the PDE in Equation (2), result in a rapid decay of its singular values towards zero [2,17]. Solution components corresponding to small singular values are strongly suppressed by the forward operator and cannot be recovered from noisy data: a small perturbation in y produces a large error in the reconstructed source. This motivates the regularized formulation stated in Equation (1).

2.2 Finite element discretization

2.2.1 The finite element method

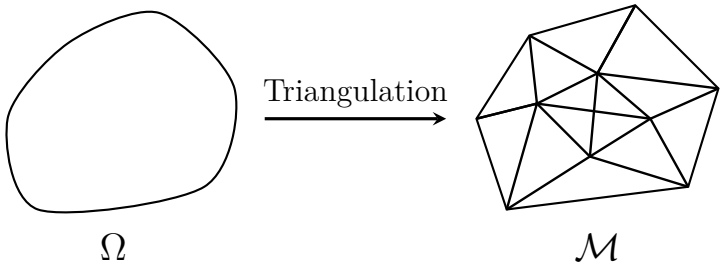


Figure 2: The triangulation $\mathcal{M}$ (the mesh) of Ω. $\mathcal{M}$ is made up of N nodes located at the vertices (corners) of the triangles.

Consider the PDE in Equation (2). Multiplying the equation by a test function $v \in H^1(\Omega)$, integrating over Ω, and applying Green's identity yields the weak formulation: find $u \in H^1(\Omega)$ such that

$$\int_\Omega (\sigma \nabla u \cdot \nabla v + cuv)\, dx = \int_\Omega fv\, dx, \text{ for all } v \in H^1(\Omega). \tag{6}$$

For details on the derivation, see Section 4 in Larson and Bengzon [18].

Let $\mathcal{M}$ be a triangulation of Ω made up of N nodes, with N_b boundary nodes, see Figure 2 as an example. Introduce the finite element space $V_h \subset H^1(\Omega)$,

$$V_h = \{v \in C^0(\Omega) : v|_M \text{ is linear for all } M \in \mathcal{M}\}. \tag{7}$$

Let $\{\phi_1, \ldots, \phi_N\}$ denote a basis for V_h, so that

$$V_h = \text{span}\{\phi_1, \ldots, \phi_N\}.$$

The basis functions ϕ_i are piecewise linear hat functions associated with the nodes of the mesh (see [18, p. 51]).

The finite element approximation of (6) then reads: find $u_h \in V_h$ such that

$$\int_\Omega (\sigma \nabla u_h \cdot \nabla v + c u_h v)\, dx = \int_\Omega f v\, dx, \text{ for all } v \in V_h. \tag{8}$$

2.2.2 The discrete forward operator

To obtain a discrete representation of the forward operator $\mathcal{K} = \mathcal{T} \circ \mathcal{S}$, we discretize both the PDE operator $\mathcal{S}$ and the trace operator $\mathcal{T}$ using the finite element space V_h.

Let $f_h, u_h \in V_h$ denote the finite element approximations of f and u, respectively. Since $V_h = \text{span}\{\phi_1, \ldots, \phi_N\}$, they can be written as

$$f_h = \sum_{j=1}^{N} x_j \phi_j, \quad u_h = \sum_{j=1}^{N} u_j \phi_j, \tag{9}$$

with coefficient vectors $x = (x_1, \ldots, x_N)^T$, $u = (u_1, \ldots, u_N)^T$.

Inserting (9) into the finite element formulation (8) and testing with the basis functions $\{\phi_i\}_{i=1}^N$ yields the linear system

$$A_h u = M x, \tag{10}$$

where $A_h \in \mathbb{R}^{N \times N}$ is the stiffness matrix and $M \in \mathbb{R}^{N \times N}$ the mass matrix, both symmetric positive definite, see Appendix A.1. Equation (10) implies that the discrete PDE operator S takes the form

$$S = A_h^{-1} M \in \mathbb{R}^{N \times N}.$$

Note that S is a mapping of coefficient vectors: mapping x to u.

The trace operator is discretized by a matrix $T \in \mathbb{R}^{N_b \times N}$ that extracts the boundary coefficients. If y denotes the vector of boundary coefficients, then

$$y = Tu.$$

Combining the discrete PDE and trace operators yields the matrix representation of the forward operator,

$$K = TS = TA_h^{-1} M \in \mathbb{R}^{N_b \times N}. \tag{11}$$

2.2.3 Discrete inner product and norm

Let V_h be the finite element space introduced in Equation (7) and let $f_h, g_h \in V_h$, with coefficient vectors $x, y \in \mathbb{R}^N$, respectively. The $L^2(\Omega)$ inner product (see Appendix A.1) of f_h and g_h, can be written as

$$\langle f_h, g_h \rangle_{L^2(\Omega)} = x^T M y,$$

where M is the mass matrix.

This induces a discrete inner product on $\mathbb{R}^N$, with associated norm, defined by

$$\langle x, y \rangle_M = x^T M y, \quad \|x\|_M = \sqrt{x^T M x}. \tag{12}$$

Similarly, let $u, v \in \mathbb{R}^{N_b}$ denote coefficient vectors corresponding to the traces $u_h|_{\partial\Omega}$ and $v_h|_{\partial\Omega}$. The $L^2(\partial\Omega)$ inner product induces a discrete inner product and norm on $\mathbb{R}^{N_b}$, given by

$$\langle u, v \rangle_{M_\partial} = u^T M_\partial v, \quad \|u\|_{M_\partial} = \sqrt{u^T M_\partial u}, \tag{13}$$

where M_∂ denotes the boundary mass matrix, see Appendix A.1.

2.2.4 The adjoint operator

Let $\mathcal{K} : L^2(\Omega) \to L^2(\partial\Omega)$ be the bounded linear operator defined in Equation (5). The adjoint operator $\mathcal{K}^* : L^2(\partial\Omega) \to L^2(\Omega)$ is the bounded linear operator satisfying

$$\langle \mathcal{K} f,\, g \rangle_{L^2(\partial\Omega)} = \langle f,\, \mathcal{K}^* g \rangle_{L^2(\Omega)}, \tag{14}$$

for all $f \in L^2(\Omega)$ and all $g \in L^2(\partial\Omega)$. Such an operator always exists for a bounded linear operator $\mathcal{K}$ mapping between two Hilbert spaces, see Section 3.9 in Kreyszig for a proof [19].

Using the definition in Equation (14), we get the weak formulation of $\mathcal{K}^*$ (the adjoint problem): find $w \in H^1(\Omega)$ such that

$$\int_\Omega (\sigma \nabla w \cdot \nabla v + c w v)\, dx = \int_{\partial\Omega} g v\, ds, \text{ for all } v \in H^1(\Omega). \tag{15}$$

The solution w then satisfies $\mathcal{K}^* g = w$. See Appendix A.2 for a complete derivation.

In the discrete case, under the inner products and norms defined in Equations (12) and (13), the adjoint operator $K^* \in \mathbb{R}^{N \times N_b}$ of $K \in \mathbb{R}^{N_b \times N}$ satisfies

$$(Kx)^T M_\partial y = x^T M K^* y, \tag{16}$$

for all $x \in \mathbb{R}^N$ and all $y \in \mathbb{R}^{N_b}$.

Inserting the explicit representation of K given in Equation (11) and using the symmetry of A_h yields the matrix representation of K^*,

$$K^* = A_h^{-1} T^T M_\partial, \tag{17}$$

where A_h is the stiffness matrix, T the discrete trace operator, and M_∂ the boundary mass matrix.

2.3 Tikhonov regularization

Let $A \in \mathbb{R}^{m \times n}$ and consider the inverse problem $Ax = y$. If A is ill-conditioned, the matrix $A^T A$ in the normal equations $A^T Ax = A^T y$ is also ill-conditioned, and small perturbations in y cause large changes in the solution. Tikhonov regularization stabilizes the problem by solving

$$\min_{x \in \mathbb{R}^n} \|Ax - y\|^2 + \lambda^2 \|Wx\|^2, \tag{18}$$

where $\lambda \geq 0$ is the regularization parameter and $W \in \mathbb{R}^{n \times n}$ is the weight matrix. The normal equations of Equation (18) are $(A^T A + \lambda^2 W^T W)x = A^T y$, so if W is diagonal with $\text{diag}(W) > 0$ and $\lambda > 0$, then $(A^T A + \lambda^2 W^T W)$ is SPD, stabilizing the problem and making the solution unique.

The weights W can include prior knowledge about the solution to penalize specific types of unwanted solutions [8]. With $W = I$, this is standard Tikhonov regularization.

2.4 SVD and low-rank approximations

The SVD of K underlies the methods developed in this thesis. Since K has a rapid decay of singular values, K can be approximated well by a low-rank matrix. Additionally, the SVD is needed to construct the pseudoinverse of K, which is used for the regularization weights in Section 2.5.

Consider the goal of finding a rank-k approximation to an $m \times n$ matrix A. We seek a matrix A_k with $\text{rank}(A_k) \leq k$ which approximates A. The problem formulation is

$$\min_{\text{rank}(A_k) \leq k} \|A - A_k\|_2, \tag{19}$$

where $\|\cdot\|_2$ is the ℓ_2 operator norm, also known as the spectral norm.

2.4.1 Singular value decomposition

The singular value decomposition of a matrix $A \in \mathbb{R}^{m \times n}$ is a factorization of the form

$$A = U \Sigma V^T,$$

where U is an $m \times m$ orthogonal matrix, Σ is an $m \times n$ diagonal matrix with non-negative entries, called the singular values of A, and V is an $n \times n$ orthogonal matrix. The singular values are ordered such that $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n$. Every matrix has such a factorization, where Σ is uniquely determined, but U and V are not [20]. The columns of U and V are called the left and right singular vectors of A, respectively, and are denoted u_i and v_i.

In 1936, C. Eckart and G. Young showed that the solution to Equation (19) is $A_k = U_k \Sigma_k V_k^T$, where U_k and V_k consist of the first k left and right singular vectors

of A, respectively, and Σ_k contains the first k singular values of A [21]. This is known as the Eckart-Young-Mirsky theorem. This implies that

$$\min_{\text{rank}(X)\leq k} \|A - X\|_2 = \sigma_{k+1}, \tag{20}$$

where σ_{k+1} is the $(k+1)$th singular value of A. Equation (20) says that no rank-k approximation of A can have an error smaller than σ_{k+1}.

2.4.2 Condition number

The condition number $\kappa = \kappa(A)$ of a matrix $A \in \mathbb{R}^{n\times n}$ measures the sensitivity of the solution $x = A^{-1}y$ to perturbations in y [8]. It is computed as

$$\kappa(A) = \frac{\sigma_1}{\sigma_n},$$

assuming A^{-1} exists. If A is singular, then $\kappa(A) = \infty$. When $\kappa(A)$ is large, then a small change in y results in a large change in the solution $x = A^{-1}y$, a typical property of ill-conditioned problems.

2.4.3 Truncated SVD

The best rank-k approximation of A given by the Eckart-Young-Mirsky theorem, $A_k = U_k\Sigma_k V_k^T$, is known as the truncated SVD (tSVD). Since it truncates the small singular values of A, it serves as a regularization technique [8]. The tSVD also provides a natural reference for the approximate methods developed in this thesis.

2.4.4 The pseudoinverse

For a matrix $A \in \mathbb{R}^{m\times n}$, the Moore-Penrose inverse A^+, often referred to as the pseudoinverse, is a generalization of the inverse of a matrix. If $A = U\Sigma V^T$ is the SVD of A, then the pseudoinverse is

$$A^+ = V\Sigma^+ U^T, \tag{21}$$

where Σ^+ is the pseudoinverse of Σ. The pseudoinverse of Σ can be computed by transposing the matrix and replacing each non-zero diagonal entry with its reciprocal [22]. The pseudoinverse A^+ is unique and always exists, and for invertible A, $A^+ = A^{-1}$.

For the least-squares problem $\min \|Ax - y\|$, the pseudoinverse provides the least-squares solution $x = A^+y$. If the least-squares solution is not unique, then A^+y is the solution with the smallest Euclidean norm. For details, see Section 5.5 in Golub and Van Loan [22].

2.4.5 Randomized SVD

The randomized SVD is a technique used to compute a rank-k approximation of an $m \times n$ matrix without computing the SVD. The core idea is to use random sampling $Y = A\Psi$ with a random matrix Ψ to find a low-rank matrix $Q = \text{orth}(Y)$ which approximates the range of A, project A onto range(Q), and compute the SVD of the projection. The projection QQ^TA will, with high probability, capture the dominant actions of A [23]. Computing $B = Q^TA$ and its SVD $B = \tilde{U}\Sigma V^T$ then gives an approximate SVD $A \approx (Q\tilde{U})\Sigma V^T$. This procedure is summarized in Algorithm 1. The basis Q is obtained via a QR factorization of Y (see Appendix A.3).

Let A_k denote a rank-k approximation of A obtained using rSVD with a random test matrix Ψ. Error analysis of rSVD is an active area of research. Halko et al. derived initial reference error analysis in terms of expectation [23], which was later improved upon by Gu [24]. Diouane et al. have recently derived further improvements [25].

Although theoretical upper bounds are useful in many applications, it is well known that randomized algorithms for low-rank approximations, like Algorithm 1 and the like, far outperform their error bounds in numerical experiments [23,24,26].

Algorithm 1 rSVD

Input: $A \in \mathbb{R}^{m\times n}$, target rank k, oversampling parameter p
Output: Approximate rank-k factorization $U\Sigma V^T$

1: Generate a random $n \times (k + p)$ test matrix Ψ
2: Compute $Y = A\Psi$
3: Form an orthonormal basis Q of Y
4: Set $B = Q^TA$
5: Compute the SVD $B = \tilde{U}\Sigma V^T$
6: Set $U = Q\tilde{U}$
7: Truncate to rank k: $U, \Sigma, V^T \leftarrow \text{truncate}_k(U, \Sigma, V^T)$

2.5 The discrete inverse problem

Replacing $\mathcal{K}$ by its discretization $K \in \mathbb{R}^{N_b\times N}$ and the L^2 norms by the FEM inner products in Equations (12) and (13), the Tikhonov problem in Equation (1) takes the discrete form

$$\min_{x\in\mathbb{R}^N} \|Kx - y\|^2_{M_\partial} + \lambda^2\|Wx\|^2_M, \tag{22}$$

where $x \in \mathbb{R}^N$ is the coefficient vector of $f_h \in V_h$, $y \in \mathbb{R}^{N_b}$ is the observed boundary data, $\lambda \geq 0$ is the regularization parameter, and W the regularization weight matrix.

Standard Tikhonov ($W = I$) does not work here. Because K has a large null space, the solutions it produces concentrate near the domain boundary regardless of where the true source is located [9]. This is illustrated in Figure 3.

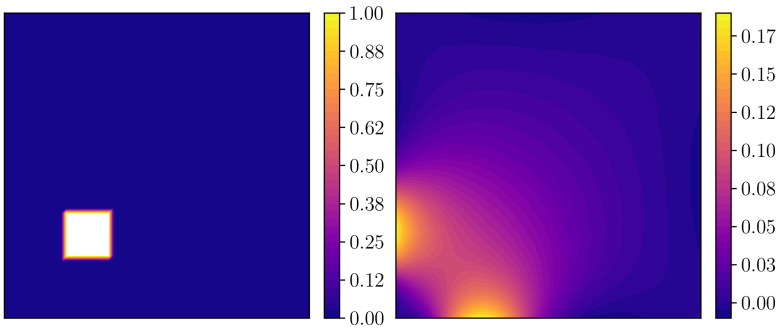


Figure 3: The true source x (left) and a standard Tikhonov solution x_λ (right), where $\lambda = 10^{-4}$.

2.5.1 Regularization weights

Assume that $\{\phi_i\}$ is an orthonormal basis of V_h, let $\mathrm{Nul}(\mathcal{K})$ denote the null space of $\mathcal{K}$ and $\mathrm{Nul}(\mathcal{K})^\perp$ its orthogonal complement. Any source $f \in V_h$ can be written as $f = f_\perp + f_0$, where $f_\perp \in \mathrm{Nul}(\mathcal{K})^\perp$ and $f_0 \in \mathrm{Nul}(\mathcal{K})$. As Elvetun and Nielsen point out in [9], sources f located near the domain boundary have a larger $f_\perp$ component, and sources deep within the domain have a larger f_0 component. If $f^* = \mathcal{K}^+ y$ denotes the least-squares minimum-norm solution, they show that for the case when $f = \phi_j$, we have that

$$f^* = \sum_i \langle \hat{P} f, \phi_i \rangle \phi_i,$$

where $\hat{P} : V_h \to \mathrm{Nul}(\mathcal{K})^\perp$ is the orthogonal projection onto $\mathrm{Nul}(\mathcal{K})^\perp$. Since $\hat{P}\phi_i = \phi_{i,\perp}$, basis functions deep in the interior have small $\|\hat{P}\phi_i\|$. By the Cauchy-Schwarz inequality, if $\|\hat{P}\phi_i\|$ is small, then $\langle \hat{P} f, \phi_i \rangle$ is small, and the corresponding component of f^* is small. This means that for any ϕ_i far away from $\partial\Omega$, the corresponding component of f^* is forced to be small, introducing bias to the solution. This issue transfers to the standard Tikhonov solution f_λ since $f_\lambda \to f^*$ as $\lambda \to 0$ [17].

Elvetun and Nielsen counteract this by re-scaling the norm of $\hat{P}\phi_i$ such that $\|\hat{P}\phi_i\| = 1$. This is achieved by setting $w_i = \|\hat{P}\phi_i\|$ with $W = \mathrm{diag}(w_i)$ in the Tikhonov formulation, equalizing the contribution of all components. In the discrete formulation, $\hat{P} = K^+ K$ is the orthogonal projection onto the complement of the null space, and the weights are computed as $w_i = \|K^+ K e_i\|$.

In our setting, the basis of the element space V_h given in Equation (7) needs to be normalized to match the orthonormality assumption made above. If $K = U\Sigma V^T$ is the SVD of K and M_{ii} is the i-th diagonal element of the mass matrix M, then the weights are computed as

$$w_i = \frac{\|K^+ K e_i\|_M}{\|\phi_i\|_M} = \frac{\|VV^T e_i\|_M}{\sqrt{M_{ii}}}, \quad i = 1, \ldots, N, \tag{23}$$

and $W = \mathrm{diag}(w_1, \ldots, w_N)$.

2.5.2 A transformed formulation

The weights in Equation (23) are defined by measuring length in the subspace $\text{Nul}(K)^{\perp} \subset \mathbb{R}^N$. However, the residual vector $Kx - y$ in Equation (22) is measured in the boundary space $\mathbb{R}^{N_b}$. Following Elvetun and Nielsen in [27], we can reformulate the equation $Kx = y$ as the equivalent problem $K^+Kx = K^+y$ (since K and K^+K share the same nullspace). Now, the residual $K^+Kx - K^+y$ lives in the subspace $\text{Nul}(K)^{\perp}$, the same space where the weights w_i are defined. Motivated by this, we replace the original residual by the transformed one, giving

$$\min_{x \in \mathbb{R}^N} \|K^+Kx - K^+y\|_M^2 + \lambda^2 \|Wx\|_M^2. \tag{24}$$

In practice, the pseudoinverse K^+ amplifies errors associated with small singular values of K, and must therefore be approximated by the truncated pseudoinverse K_k^+, defined using the k largest singular vectors in Equation (21). The weights are correspondingly approximated using $P_k = K_k^+ K$.

3 Approximating the forward operator

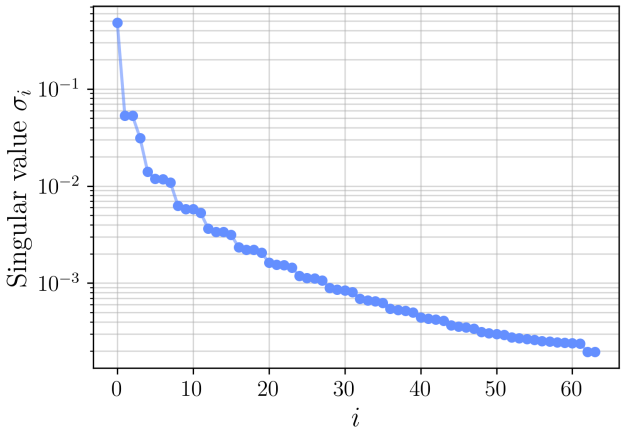


Figure 4: The rapidly decaying singular values $\sigma_1, \ldots, \sigma_{64}$ of a 64×289 matrix K constructed from a 16×16 rectangular mesh. The y-axis uses a log scale.

The goal of this section is to compute the regularization weights given in Equation (23), which requires the SVD $K = U\Sigma V^T$. Using the identity $K^T = MA_h^{-1}T^T$, K can be formed with N_b linear solves of $A_h v_i = T^T e_i$. With a sparse Cholesky factorization of A_h at cost $\mathcal{O}(N^{3/2})$ and triangular solves at $\mathcal{O}(N \log N)$ in two dimensions (Appendix A.4), forming K takes $\mathcal{O}(N_b N \log N)$. The SVD of the resulting dense $N_b \times N$ matrix then costs $\mathcal{O}(N_b^2 N)$ [28]. Since N grows quadratically with mesh resolution in two dimensions and cubically in three, forming K and computing its SVD is infeasible at fine meshes.

The randomized SVD (Algorithm 1) reduces this cost to $\mathcal{O}(lN_bN)$ [23], where $l = k+p$ is a small constant, k is the target rank, and p is an oversampling parameter (typically, $p \in [5, 10]$ [23]). It yields a rank-k approximation $K \approx U\Sigma V^T$, and since K has a large nullspace and rapidly decaying singular values, as illustrated in Figure 4, l can be chosen much smaller than N_b. Unlike $N_b \sim \sqrt{N}$, l can be held constant when the mesh is refined (as shown in Section 5.3.5), so the ratio l/N_b shrinks as N increases and the rSVD becomes increasingly advantageous for larger problems.

The rSVD implementation of Algorithm 1 requires forming $B = Q^T K$, which requires K to be stored as a dense $N_b \times N$ matrix. We instead propose a matrix-free variant without assembling K, at a cost of l forward PDE solves and l adjoint PDE solves.

3.1 A matrix-free rSVD scheme

The algorithm consists of two stages. The first stage constructs an approximate basis Q for the column space of K. We select a target rank k and an oversampling parameter p, set $l = k + p$, and apply K to l random test vectors $\psi_1, \psi_2, \ldots, \psi_l$ drawn from a distribution of choice, typically the standard normal distribution [23].

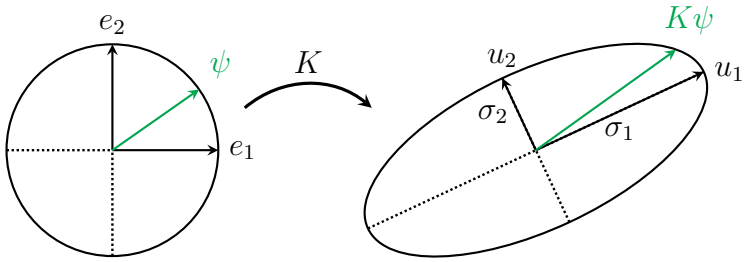


Figure 5: The geometry of the rSVD: A random sample $K\psi$ aligns mostly with the dominant left singular vectors of K. The illustration is inspired by Figure 9.1 in Tropp [29].

This yields l random samples $y_1, y_2, \ldots, y_l$, where $y_i = K\psi_i$. Collecting these into matrices $Y = [y_i]_{i=1}^l$ and $\Psi = [\psi_i]_{i=1}^l$, we have

$$Y = K\Psi.$$

Each sample $K\psi_i$ is a linear combination of the columns of K, and tends to align with the dominant left singular vectors, see Figure 5. Orthogonalizing the columns of Y via a QR-factorization $QR = Y$ ensures that $\text{span}\{q_1, \ldots, q_l\}$ approximately aligns with $\text{span}\{u_1, \ldots, u_l\}$, giving

$$\text{range}(Q) \approx \text{range}(K).$$

The second stage projects K onto $\text{range}(Q)$ by computing $B = Q^T K$. Since we do not know K explicitly, we use K^* to compute B. In particular, by the definition of K^*,

$$(K^* z)^T M = z^T M_\partial K. \tag{25}$$

If we choose $z = M_\partial^{-1} q_i$ and insert it into Equation (25), we see that

$$(K^* z)^T M = (M_\partial^{-1} q_i)^T M_\partial K = q_i^T K. \tag{26}$$

Setting $b_i = (K^*(M_\partial^{-1} q_i))^T M$ for $i = 1, 2, \ldots, l$, and defining the $l \times N$ matrix

$$B = \begin{bmatrix} b_1 \\ \vdots \\ b_l \end{bmatrix},$$

we obtain, by Equation (26),

$$B = Q^T K.$$

Taking the SVD $B = \tilde{U}\Sigma V^T$, and setting $U = Q\tilde{U}$, gives an approximate SVD

$$K \approx U\Sigma V^T. \tag{27}$$

The factorization in (27) is of rank l, since $U\Sigma V^T = Q\tilde{U}\Sigma V^T = QB = QQ^T K$ with $\text{rank}(Q) = l$. A rank-k factorization is obtained by selecting the k first columns of the respective matrices in (27). The procedure is summarized in Algorithm 2.

Algorithm 2 Matrix-free rSVD approximation of K

Input: Forward operator $K : \mathbb{R}^N \to \mathbb{R}^{N_b}$, adjoint operator $K^* : \mathbb{R}^{N_b} \to \mathbb{R}^N$, mass matrices M, M_∂, target rank k, oversampling parameter p

Output: Approximate rank-k factorization $K \approx U\Sigma V^T$

Range sketch:
Set $l = k + p$ ▷ Ensure that $l \leq N_b$
for $i = 1, \ldots, l$ **do**
    Draw a random vector $\psi_i \in \mathbb{R}^N$
    Apply the forward operator: $y_i = K\psi_i \in \mathbb{R}^{N_b}$
end for
Form the matrix $Y = [y_1, \ldots, y_l] \in \mathbb{R}^{N_b \times l}$ ▷ Equivalent to $Y = K\Psi$
Compute an orthonormal basis: $Q = \mathrm{orth}(Y)$
Projection onto the basis:
for $i = 1, \ldots, l$ **do**
    Let q_i be the i-th column of Q
    Solve for $\tilde{q}_i = M_\partial^{-1} q_i$
    Apply the adjoint operator: $\tilde{b}_i = K^* \tilde{q}_i$
    Set $b_i = \tilde{b}_i^T M$ ▷ i-th row of B
end for
Form the projected matrix $B = \begin{bmatrix} b_1 \\ \vdots \\ b_l \end{bmatrix} \in \mathbb{R}^{l \times N}$ ▷ Equivalent to $B = Q^T K$
Compute the SVD: $B = \tilde{U}\Sigma V^T$
Set $U = Q\tilde{U}$
Truncate to rank k: $U, \Sigma, V^T \leftarrow \mathrm{truncate}_k(U, \Sigma, V^T)$

3.2 Approximating the adjoint operator

An alternative is to apply the rSVD to the adjoint operator K^*. By Equation (16), $K = M_\partial^{-1}(K^*)^T M$, so if $K^* \approx \bar{U}\bar{\Sigma}\bar{V}^T$ is an approximation obtained via the rSVD, then K can be recovered via $K = M_\partial^{-1}\bar{V}\bar{\Sigma}\bar{U}^T M$. In practice, this variant may yield a better low-rank approximation, depending on the quality of the random sampling $K^*\psi_i$ compared to $K\psi_j$.

Algorithm 2 extends directly to K^* by replacing K with K^* and drawing random test vectors $\psi_i \in \mathbb{R}^{N_b}$, obtaining a sampling matrix $Y \in \mathbb{R}^{N\times l}$ and an orthonormal basis $Q = \mathrm{orth}(Y)$. For the projection step $B = Q^T K^*$, choosing $z = M^{-1}q_i$ in the adjoint identity $(Kz)^T M_\partial = z^T M K^*$ gives $b_i = (K(M^{-1}q_i))^T M_\partial$, by the same argument as in Section 3.1. The procedure is summarized in Algorithm 3.

Algorithm 3 Matrix-free rSVD approximation of K^*

Input: Forward operator $K : \mathbb{R}^N \to \mathbb{R}^{N_b}$, adjoint operator $K^* : \mathbb{R}^{N_b} \to \mathbb{R}^N$, mass matrices M, M_∂, target rank k, oversampling parameter p

Output: Approximate rank-k factorization $K^* \approx \bar{U}\bar{\Sigma}\bar{V}^T$

Range sketch:
Set $l = k + p$ ▷ Ensure that $l \le N_b$
for $i = 1, \dots, l$ **do**
    Draw a random vector $\psi_i \in \mathbb{R}^{N_b}$
    Apply the adjoint operator: $y_i = K^*\psi_i \in \mathbb{R}^N$
end for
Form the matrix $Y = [y_1, \dots, y_l] \in \mathbb{R}^{N\times l}$ ▷ Equivalent to $Y = K^*\Psi$
Compute an orthonormal basis: $Q = \mathrm{orth}(Y)$
Projection onto the basis:
for $i = 1, \dots, l$ **do**
    Let q_i be the i-th column of Q
    Solve for $\tilde{q}_i = M^{-1}q_i$
    Apply the forward operator: $\tilde{b}_i = K\tilde{q}_i \in \mathbb{R}^{N_b}$
    Set $b_i = \tilde{b}_i^T M_\partial$ ▷ i-th row of B
end for
Form the projected matrix $B = \begin{bmatrix} b_1 \\ \vdots \\ b_l \end{bmatrix} \in \mathbb{R}^{l\times N_b}$ ▷ Equivalent to $B = Q^T K^*$
Compute the SVD: $B = \tilde{U}\bar{\Sigma}\bar{V}^T$
Set $\bar{U} = Q\tilde{U}$
Truncate to rank k: $\bar{U}, \bar{\Sigma}, \bar{V}^T \leftarrow \mathrm{truncate}_k(\bar{U}, \bar{\Sigma}, \bar{V}^T)$

3.2.1 Recovering the SVD

Given the approximate rank-k SVD $K^* \approx \bar{U}\bar{\Sigma}\bar{V}^T$, where $\bar{U} \in \mathbb{R}^{N\times k}$, $\bar{\Sigma} \in \mathbb{R}^{k\times k}$, and $\bar{V} \in \mathbb{R}^{N_b\times k}$, K can be approximated from the relation $K = M_\partial^{-1}\bar{V}\bar{\Sigma}\bar{U}^T M$. We recover the SVD of K: define the $N_b \times k$ matrix $A = M_\partial^{-1}\bar{V}$ and the $N \times k$ matrix $B = M\bar{U}$, giving the rank-k factorization

$$K \approx A\bar{\Sigma}B^T.$$

Next, compute the thin QR factorization $Q_A R_A = A$ and $Q_B R_B = B$, where $Q_A \in \mathbb{R}^{N_b\times k}$, $Q_B \in \mathbb{R}^{N\times k}$, and $R_A, R_B \in \mathbb{R}^{k\times k}$. Then, $K \approx Q_A(R_A\bar{\Sigma}R_B^T)Q_B^T$, and taking the SVD of the small $k \times k$ matrix $R_A\bar{\Sigma}R_B^T = \hat{U}\Sigma\hat{V}^T$ gives

$$K \approx Q_A(\hat{U}\Sigma\hat{V}^T)Q_B^T.$$

By setting $U = Q_A\hat{U}$ and $V = Q_B\hat{V}$, we get the rank-k approximate SVD

$$K \approx U\Sigma V^T.$$

3.3 Initial matrix factorizations

Both algorithms factorize the stiffness matrix A_h and the mass matrices M and M_∂ once at initialization. Each application of K or K^* then reduces to triangular solves with A_h, and each mass matrix solve $M_\partial\tilde{q}_i = q_i$ or $M\tilde{q}_i = q_i$ reduces to triangular solves with the respective factorization.

The matrices A_h, M and M_∂ are SPD (see Appendix A.1), and can be factorized using the Cholesky decomposition. The sparse structure of the stiffness and mass matrices can be exploited to efficiently compute such factorizations. For instance, the nested dissection method, proposed by George [30], may compute such a Cholesky factorization in $\mathcal{O}(N^{3/2})$ computations. The algorithm minimizes fill-ins, keeping the resulting triangular matrices sparse. As a consequence, once the factorization is done, solving the triangular systems can be done at a cost of $\mathcal{O}(N \log N)$ at each iteration [31,32]. For details, see Appendix A.4.

In particular, if $A_h = LL^T$ is a Cholesky factorization, then $K = T(LL^T)^{-1}M$. To compute $y_i = K\psi_i$, the system $LL^T v = M\psi_i$ is solved via two triangular solves, and $y_i = Tv$. A similar procedure is done when applying $K^* = A_h^{-1}T^T M_\partial$, where the factorization $A_h = LL^T$ can be reused since $A_h = A_h^T$.

4 Dynamical low-rank approximation

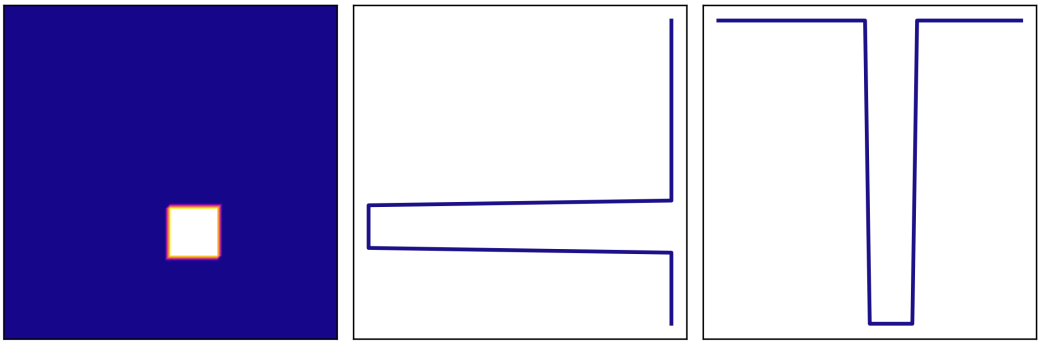


Figure 6: Rank-1 outer product representation of a single square source (left), $X = \sigma_1 u_1 v_1^T$. The left singular vector u_1 (middle) is plotted along the vertical axis, and the right singular vector v_1 (right) along the horizontal axis.

In many applications, the source $f \in L^2(\Omega)$ in Equation (1) is sparse [27], and often well-approximated by low-rank matrices. If $x \in \mathbb{R}^{n^2}$ is a discretization of f and $X = \mathrm{mat}(x)$ is its reordering into an $n \times n$ matrix, we can often expect $r = \mathrm{rank}(X) \ll n$. The SVD writes X as a sum of r outer products,

$$X = \sum_{i=1}^{r} \sigma_i u_i v_i^T,$$

each parameterized by two vectors $u_i, v_i \in \mathbb{R}^n$ (Figure 6), with r roughly tracking the number of distinct localized sources. Since the source itself is low-rank, we restrict the search to rank-r matrices, working directly with the factors u_i, v_i rather than the full matrix X.

4.1 The low-rank manifold

Consider the problem of finding a rank-r solution X via the update scheme $X^{(i+1)} = X^{(i)} + \alpha_i D^{(i)}$, where $D^{(i)}$ is a search direction and $\alpha_i > 0$ a step size. Starting from a rank-r matrix $X^{(0)}$, there is in general no guarantee that $D^{(i)}$ preserves low rank, so that $X^{(i)}$ may leave the space $\mathcal{M}_r$ of rank-r matrices after a single step.

One can replace $D^{(i)}$ by its projection onto the tangent space $\mathcal{T}_{X^{(i)}}(\mathcal{M}_r)$, a classical concept from Riemannian geometry [33,34]. This is the idea behind the Dynamical Low-Rank Approximation (DLRA) of Koch and Lubich [35]. They proposed, in a continuous-time setting, evolving an approximation on $\mathcal{M}_r$ by projecting the derivative of the matrix onto the tangent space at the current approximation (see Appendix A.5 for details). Using this concept in our discrete setting, we replace $D^{(i)}$ by its tangent-space projection $\mathcal{P}_{X^{(i)}}(D^{(i)})$, yielding the update

$$X^{(i+1)} = X^{(i)} + \alpha_i \mathcal{P}_{X^{(i)}}(D^{(i)}).$$

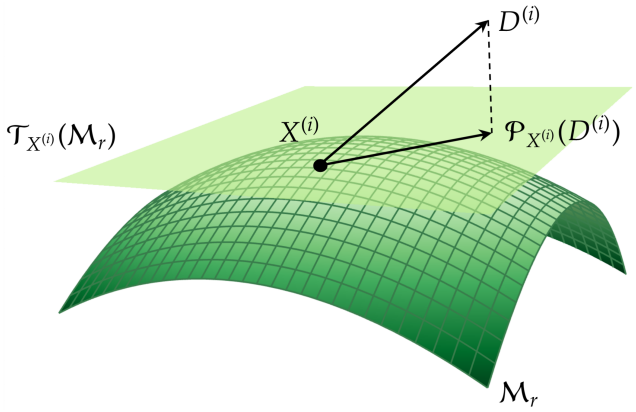


Figure 7: Illustration of the DLRA algorithm: the point $X^{(i)}$ lies on the manifold of rank-r matrices $\mathcal{M}_r$, and $D^{(i)}$ is projected onto the tangent plane $\mathcal{T}_{X^{(i)}}(\mathcal{M}_r)$.

This is illustrated in Figure 7. Rather than updating $X^{(i)}$ directly, the DLRA algorithm operates on a factorization $X^{(i)} = U_x \Sigma_x V_x^T$, where U_x and V_x have orthonormal columns and Σ_x is non-singular (and diagonal if using the SVD), updating the factors at each iteration. We use the subscript "x" to distinguish these factors from those of the forward operator $K = U\Sigma V^T$.

However, as pointed out by Koch and Lubich, when Σ_x is ill-conditioned, which occurs when r exceeds the effective rank of $X^{(i)}$, the projector $\mathcal{P}_{X^{(i)}}$ suffers from stiffness, leading to very small step sizes and issues with rounding errors.

To address this, Lubich and Oseledets proposed splitting the projector $\mathcal{P}_{X^{(i)}}$ into simpler sub-projections [36]. Crucially, each sub-projection can be evaluated without forming Σ_x^{-1}, and avoids the stiffness problem present in the original formulation [37]. In recent years, additional advancements have been made to DLRA, offering improved stability [38], parallelism [39], and rank-adaptivity [40].

Schotthöfer et al. [11] recently proposed a discrete rank-adaptive update scheme based on the "unconventional" basis update and Galerkin step integrator [41] for training low-rank neural networks, see Algorithm 4. The discrete DLRA scheme used in this thesis follows their formulation.

4.2 Step-size selection and rank adaptation

Algorithm 4 admits several choices for the step size α_i and the truncation criterion.

Step size. We consider three step-size rules. A fixed step size $\alpha_i = \alpha > 0$ coincides with one step of gradient descent on the factors K', L', and $\hat{\Sigma}$, analogous to the stochastic gradient descent (SGD) step used by Schotthöfer et al. [11], but using the full gradient. Adam computes adaptive step sizes from estimates of the first and second moments of the gradient [12], and Schotthöfer et al. find it converges faster in practice than SGD for training neural networks [11]. More recently, they showed

Algorithm 4 Schotthöfer et al.'s DLRA scheme

Input: Initial matrix $X^{(0)}$ with SVD $X^{(0)} = U_x \Sigma_x V_x^T$, solution rank r, step-size rule $\{\alpha_i\}$

Output: Rank-r solution $x = \text{vec}(X)$

1: **for** $i = 0, 1, 2, \ldots, \text{max\_iter}$ **do**
2: **Search direction:**
3: $D^{(i)} = -\nabla\Phi(X^{(i)})$
4: **Dynamical low-rank update (KLS step):**
5: $K' = U_x\Sigma_x + \alpha_i D^{(i)} V_x$
6: $L' = V_x\Sigma_x^T + \alpha_i (D^{(i)})^T U_x$
7: $\hat{U} = \text{orth}([U_x, K'])$
8: $\hat{V} = \text{orth}([V_x, L'])$
9: $\hat{\Sigma} = (\hat{U}^T U_x)\Sigma_x (V_x^T \hat{V}) + \alpha_i (\hat{U}^T D^{(i)} \hat{V})$
10: $(U_x, \Sigma_x, V_x) = \text{truncate}_r(\hat{U}, \hat{\Sigma}, \hat{V})$ ▷ $X^{(i+1)} = U_x \Sigma_x V_x^T$
11: **end for**

that classical momentum methods like Adam can struggle due to the geometry of $\mathcal{M}_r$ [42].

For the quadratic objective Φ considered in this thesis (Equation (22)), steepest descent selects the step size that minimizes Φ along $D^{(i)} = -\nabla\Phi(X^{(i)})$,

$$\alpha_i = \frac{\|\nabla\Phi(X^{(i)})\|_F^2}{\langle D^{(i)},\, HD^{(i)}\rangle_F},$$

and requires no tuning (Section 8.2 in Chong et al. [13]).

Rank adaptation. In the dynamical low-rank update (KLS) step of Algorithm 4, the factors $\hat{U}$ and $\hat{V}$ have $2r$ columns and $\hat{\Sigma}$ is of shape $2r \times 2r$. The truncation step (line 10) moves the factors back to a specified maximum rank r [11]. The truncation can also be done adaptively via a truncation threshold ϑ, as done by Schotthöfer et al. [11].

4.3 Application to the inverse problem

In the following sections, we consider the problem of minimizing a quadratic objective function

$$\Phi : \mathbb{R}^{n\times n} \to \mathbb{R}.$$

In our discrete case, the minimization problem given in Equation (22) translates into minimizing

$$\Phi(X) = \frac{1}{2}\|Kx - y\|_{M_\partial}^2 + \frac{\lambda^2}{2}\|Wx\|_M^2.$$

Using the product rule, we get the gradient $\nabla\Phi$ and Hessian H of Φ with respect to x,

$$\nabla\Phi(X) = K^T M_\partial (Kx - y) + \lambda^2 W^T M W x,$$
$$H = K^T M_\partial K + \lambda^2 W^T M W.$$

The Hessian is SPD, since $K^T M_\partial K$ is positive semidefinite, $\lambda^2 W^T M W$ is positive definite, and their sum is therefore positive definite. By the optimality condition $\nabla\Phi(X) = 0$, the minimizer of Φ is the solution of $Hx = K^T M_\partial y$, i.e., the normal equations of Equation (22).

4.4 DLRA with conjugate search directions

Since Φ is quadratic with an SPD Hessian H, the conjugate gradient (CG) method is a natural minimization strategy [43,16,15], see Appendix A.6.

CG generates a sequence of search directions $\{D^{(i)}\}$ that are H-conjugate, i.e., $\langle D^{(i)}, HD^{(j)}\rangle_F = 0$ for $i \neq j$. Starting from the initial search direction $D^{(0)} = -G^{(0)}$, each step computes an optimal step size

$$\alpha_i = \frac{\|G^{(i)}\|_F^2}{\langle D^{(i)},\, HD^{(i)}\rangle_F},$$

and updates the search direction by the Fletcher-Reeves rule [16],

$$D^{(i+1)} = -G^{(i+1)} + \beta_i D^{(i)}, \qquad \beta_i = \frac{\|G^{(i+1)}\|_F^2}{\|G^{(i)}\|_F^2}. \tag{28}$$

In our low-rank setting, we propose replacing the descent directions used in the method of Schotthöfer et al. (Algorithm 4) with H-conjugate search directions $\{D^{(i)}\}$ obtained via the CG method. The motivation is that H-conjugate directions exploit curvature information of H, leading to faster convergence. The full scheme is given in Algorithm 5, which we refer to as DLRA-CG throughout.

4.4.1 Restarted CG

In Algorithm 5, the sequence $\{D^{(i)}\}$ is generated independently of the manifold iterates $X^{(i)}$. After the first step, $G^{(i)}$ no longer equals $\nabla\Phi(X^{(i)})$: the recurrence $G^{(i+1)} = G^{(i)} + \alpha_i H D^{(i)}$ tracks the gradient along the full-rank trajectory

$$Y^{(i)} = X^{(0)} + \sum_{j<i} \alpha_j D^{(j)}, \tag{29}$$

not along the manifold iterates. As $Y^{(i)}$ and $X^{(i)}$ drift apart, $G^{(i)} = \nabla\Phi(Y^{(i)}) \to 0$ as plain CG converges along $Y^{(i)}$, so $\alpha_i \to 0$ and $X^{(i)}$ may stall at a sub-optimal solution.

Restarted CG [44] re-computes the true gradient $G^{(i)} = \nabla\Phi(X^{(i)})$ and resets the search direction to $D^{(i)} = -G^{(i)}$ every N_r iterations, discarding the previous search directions. This was originally proposed as a method to counteract the loss

Algorithm 5 DLRA-CG scheme

Input: Hessian H, initial matrix $X^{(0)}$ with SVD $X^{(0)} = U_x \Sigma_x V_x^T$, solution rank r

Output: Rank-r solution $x = \mathrm{vec}(X)$

Compute initial gradient $G^{(0)} = \nabla \Phi(X^{(0)})$

Set initial search direction $D^{(0)} = -G^{(0)}$

for $i = 0, 1, 2, \ldots, \mathrm{max\_iter}$ **do**

  Step size:

  $\alpha_i = \dfrac{\|G^{(i)}\|_F^2}{\langle D^{(i)}, HD^{(i)} \rangle_F}$ ▷ $HD^{(i)} = \mathrm{mat}[H\,\mathrm{vec}(D^{(i)})]$

  Dynamical low-rank update (KLS step):

  $K' = U_x \Sigma_x + \alpha_i D^{(i)} V_x$

  $L' = V_x \Sigma_x^T + \alpha_i (D^{(i)})^T U_x$

  $\hat{U} = \mathrm{orth}([U_x, K'])$

  $\hat{V} = \mathrm{orth}([V_x, L'])$

  $\hat{\Sigma} = (\hat{U}^T U_x) \Sigma_x (V_x^T \hat{V}) + \alpha_i (\hat{U}^T D^{(i)} \hat{V})$

  $(U_x, \Sigma_x, V_x) = \mathrm{truncate}_r(\hat{U}, \hat{\Sigma}, \hat{V})$ ▷ $X^{(i+1)} = U_x \Sigma_x V_x^T$

  Search direction update:

  $G^{(i+1)} = G^{(i)} + \alpha_i H D^{(i)}$

  $\beta_i = \dfrac{\|G^{(i+1)}\|_F^2}{\|G^{(i)}\|_F^2}$

  $D^{(i+1)} = -G^{(i+1)} + \beta_i D^{(i)}$

end for

of H-conjugacy in the construction of $\{D^{(i)}\}$ because of numerical round-off errors [15,44]. In the DLRA-CG setting, restart serves a second purpose beyond correcting round-off: it resets the full-rank trajectory to $Y^{(i)} = X^{(i)}$, restoring $G^{(i)} = \nabla\Phi(X^{(i)})$ and giving the CG recurrence a fresh start from the true iterate position $X^{(i)}$.

In Algorithm 5, this replaces the recurrence $G^{(i+1)} = G^{(i)} + \alpha_i H D^{(i)}$ with a fresh gradient evaluation $G^{(i+1)} = \nabla\Phi(X^{(i+1)})$, and sets $D^{(i+1)} = -G^{(i+1)}$, skipping the β_i step.

4.5 DLRA using preconditioned CG

Preconditioning is a standard technique for accelerating CG methods, referred to as preconditioned CG (PCG). The system $Hx = K^T M_\partial y$ is replaced by the equivalent system $P^{-1}Hx = P^{-1}K^T M_\partial y$, where $P \approx H$ is the preconditioner matrix. The convergence of CG depends on $\kappa(H)$ (see Section 6.11.3 in Saad [43]), so P is chosen such that $\kappa(P^{-1}H) \ll \kappa(H)$. For elliptic PDE-based forward operators K, the Hessian $H = K^T M_\partial K + \lambda^2 W^T M W$ typically has a large condition number, making preconditioning effective.

In the DLRA-CG scheme, the preconditioned system $P^{-1}Hx = P^{-1}K^T M_\partial y$ leads to the following adjustments in Algorithm 5:

$$\alpha_i = \frac{\langle G^{(i)}, Z^{(i)}\rangle_F}{\langle D^{(i)}, HD^{(i)}\rangle_F}, \quad \beta_i = \frac{\langle G^{(i+1)}, Z^{(i+1)}\rangle_F}{\langle G^{(i)}, Z^{(i)}\rangle_F}, \quad D^{(i+1)} = -Z^{(i+1)} + \beta_i D^{(i)},$$

with $Z^{(i)} = P^{-1}G^{(i)}$. These modifications are equivalent to standard preconditioned conjugate gradient, as originally described by Concus et al. [45]. The resulting scheme is summarized in Algorithm 6, which we refer to as DLRA-PCG.

4.5.1 Choice of preconditioner

The effectiveness of PCG depends on the choice of P. A good preconditioner must approximate H well, but must be cheap to apply at each iteration. We consider four preconditioners: Jacobi scaling, incomplete Cholesky (IC), IC with a Woodbury correction, and a "perfect" preconditioner based on a sparse Cholesky factorization.

Jacobi scaling. A Jacobi preconditioner $P = \mathrm{diag}(H)$ approximates H by its diagonal (Section 11.5, [22]). The approximation is cheap to compute, as P^{-1} is the inverse of a diagonal matrix. It is most effective when H is diagonally dominant.

Incomplete Cholesky. Under the assumption that K is of rank k (for example, obtained using Algorithm 2), the Hessian consists of a low-rank plus a sparse term,

$$H = \underbrace{K^T M_\partial K}_{\text{low-rank}} + \underbrace{\lambda^2 W^T M W}_{\text{sparse}} = F + S.$$

Algorithm 6 DLRA-PCG scheme

Input: Hessian H, initial matrix $X^{(0)}$ with SVD $X^{(0)} = U_x \Sigma_x V_x^T$, solution rank r, preconditioner P^{-1}

Output: Rank-r solution $x = \mathrm{vec}(X)$

Compute initial gradient $G^{(0)} = \nabla \Phi(X^{(0)})$

Precondition $Z^{(0)} = P^{-1} G^{(0)}$ ▷ $P^{-1} G^{(0)} = \mathrm{mat}[P^{-1} \mathrm{vec}(G^{(0)})]$

Set initial search direction $D^{(0)} = -Z^{(0)}$

for $i = 0, 1, 2, \ldots, \mathrm{max\_iter}$ **do**

  Step size:

  $\alpha_i = \dfrac{\langle G^{(i)}, Z^{(i)} \rangle_F}{\langle D^{(i)}, H D^{(i)} \rangle_F}$ ▷ $H D^{(i)} = \mathrm{mat}[H \mathrm{vec}(D^{(i)})]$

  Dynamical low-rank update (KLS step):

  $K' = U_x \Sigma_x + \alpha_i D^{(i)} V_x$

  $L' = V_x \Sigma_x^T + \alpha_i (D^{(i)})^T U_x$

  $\hat{U} = \mathrm{orth}([U_x, K'])$

  $\hat{V} = \mathrm{orth}([V_x, L'])$

  $\hat{\Sigma} = (\hat{U}^T U_x) \Sigma_x (V_x^T \hat{V}) + \alpha_i (\hat{U}^T D^{(i)} \hat{V})$

  $(U_x, \Sigma_x, V_x) = \mathrm{truncate}_r(\hat{U}, \hat{\Sigma}, \hat{V})$ ▷ $X^{(i+1)} = U_x \Sigma_x V_x^T$

  Search direction update:

  $G^{(i+1)} = G^{(i)} + \alpha_i H D^{(i)}$

  $Z^{(i+1)} = P^{-1} G^{(i+1)}$

  $\beta_i = \dfrac{\langle G^{(i+1)}, Z^{(i+1)} \rangle_F}{\langle G^{(i)}, Z^{(i)} \rangle_F}$

  $D^{(i+1)} = -Z^{(i+1)} + \beta_i D^{(i)}$

end for

To see why, insert the rank-k SVD of K into $F = K^T M_\partial K$ to get $V\Sigma U^T M_\partial U \Sigma V^T$. Since $U^T M_\partial U \in \mathbb{R}^{k\times k}$ is SPD, it has full rank k, and hence F has rank k. The term $S = \lambda^2 W^T M W$ is sparse since it is just a scaling of M, which itself is sparse.

Since S is SPD, it can effectively be approximated using an incomplete Cholesky factorization $S \approx LL^T$, where L is lower-triangular (Section 11.5, [22]). The preconditioner can then be set as $P^{-1} = L^{-T}L^{-1}$. This preconditioner works well when the sparse part approximates the Hessian well ($H \approx S$).

IC with Woodbury correction. To account for the low-rank part of H, we can use the Woodbury formula [46] as a correction. Using the SVD of K, we write $F = (V\Sigma)U^T M_\partial U (V\Sigma)^T = \tilde{V} C \tilde{V}^T$ where $\tilde{V} = V\Sigma$ and $C = U^T M_\partial U$. By the Woodbury formula,

$$(S + \tilde{V} C \tilde{V}^T)^{-1} = S^{-1} - S^{-1}\tilde{V}(C^{-1} + \tilde{V}^T S^{-1} \tilde{V})^{-1} \tilde{V}^T S^{-1}.$$

Replacing S with an incomplete Cholesky factorization LL^T gives the preconditioner.

Perfect preconditioner. If we replace the incomplete Cholesky factorization LL^T with an exact sparse Cholesky factorization of S, we obtain a "perfect" preconditioner with $P^{-1} = H^{-1}$. For standard PCG, this would be equivalent to solving the normal equations $Hx = K^T M_\partial y$ directly in a single iteration. In the DLRA-PCG setting, however, a truncation is performed at each iteration, so convergence in a single step is not guaranteed.

5 Results

5.1 Experimental setup

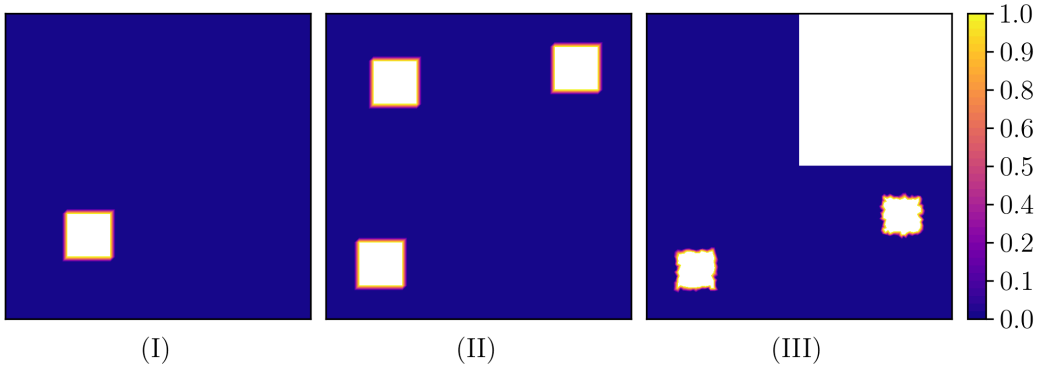


Figure 8: The test problems used for the results: Problem I (left), Problem II (middle), and Problem III (right).

Figure 8 shows the three meshes and source functions used throughout the results. Problem I is a square mesh with a single source, used as a baseline. Problem II uses the same geometry with three sources to assess reconstruction under multiple sources. Problem III replaces the square domain with an "L"-shaped mesh and two sources, introducing a non-convex geometry. Problems I and II use a uniform grid, whereas Problem III uses a non-uniform grid.

For all experiments, we set the diffusion coefficient $\sigma = 1$ and the reaction coefficient $c = 1$ in Equation (2). The finite element space V_h (Equation (7)) uses a basis of continuous piecewise-linear hat functions ϕ_i (continuous Galerkin, degree 1).

Unless stated otherwise, we use $N = 64^2 = 4096$ for Problems I and II and $N = 6127$ for Problem III, regularization parameter $\lambda = 10^{-4}$, oversampling $p = 5$, and Gaussian random test vectors ψ_i for the rSVD.

5.2 Computational setup

All numerical experiments were performed on a laptop running Fedora Linux 43 with Linux kernel version 6.19. The system was running with an Intel(R) Core(TM) i7-10510U (4 cores, 8 threads) with 16GB of memory.

Python 3.9 was used for all computations, together with NumPy and SciPy for linear algebra routines [47,48]. The FEniCS software was used for solving PDEs using the finite element method [49,50]. All library specifications, algorithm implementations, and notebooks used to generate the results in this thesis are openly available at: github.com/EliasSev/masters-thesis.

5.3 Matrix-free rSVD

5.3.1 Computational time

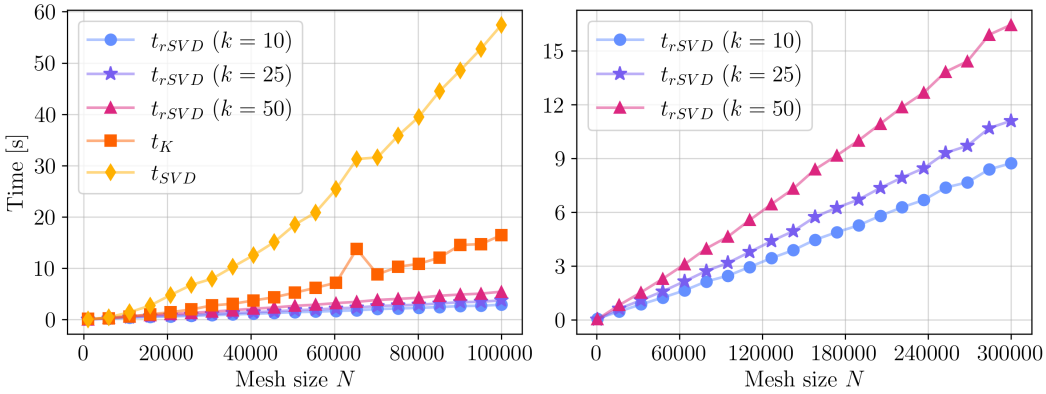


Figure 9: Computational time of using Algorithm 2 (t_{rSVD}), explicitly forming K (t_K), and computing the SVD of K (t_{SVD}) for increasing N. Forming K and computing its SVD are infeasible for large N, so t_K and t_{SVD} are omitted from the right plot. Median values over 30 runs, measured in seconds.

N	$\frac{t_{\text{rSVD}}^{(25)}}{t_{\text{rSVD}}^{(10)}}$	$\frac{t_{\text{rSVD}}^{(50)}}{t_{\text{rSVD}}^{(10)}}$	$\frac{t_K+t_{\text{SVD}}}{t_{\text{rSVD}}^{(10)}}$	$\frac{t_K+t_{\text{SVD}}}{t_{\text{rSVD}}^{(25)}}$	$\frac{t_K+t_{\text{SVD}}}{t_{\text{rSVD}}^{(50)}}$
1,000	0.79	1.14	4.21	5.31	3.68
25,750	1.29	1.77	11.81	9.13	6.68
50,500	1.26	1.85	16.21	12.84	8.78
75,250	1.27	1.85	21.13	16.61	11.39
100,000	1.29	1.87	25.47	19.76	13.62

Table 1: Computational time ratios: $t_{\text{rSVD}}^{(k)}$ at $k = 25$ and $k = 50$ relative to $k = 10$, and $(t_K + t_{\text{SVD}})$ relative to $t_{\text{rSVD}}^{(k)}$ for $k = 10, 25, 50$. Medians over 30 runs.

The left plot of Figure 9 shows the computational time of Algorithm 2 for target ranks $k = 10$, 25 and 50, compared to forming $K = TA_h^{-1}M$ explicitly (Section 3) and taking its SVD. Table 1 shows the relative speedup comparing different methods, with notation $t_{\text{rSVD}}^{(k)}$ for the time of using Algorithm 2 at target rank k.

The time t_K surpasses t_{rSVD} for $N > 1000$ across the selected target ranks, and t_{SVD} surpasses t_{rSVD} for $N > 5000$. The time t_K increases more rapidly than t_{rSVD} for all target ranks, and t_{SVD} is observed to increase faster than t_K, matching their respective theoretical costs of $\mathcal{O}(N_b^2 N)$ and $\mathcal{O}(N_b N \log N)$, established in Section 3.

The computational times t_{rSVD} for target ranks 10, 25 and 50 are seen to scale near linearly on this range of N (right plot of Figure 9). The rate of increase depends on the target rank, with a higher target rank resulting in a more rapid increase. For $N > 10{,}000$, increasing k from 10 to 50 results in an increase of t_{rSVD} by a factor

of approximately 1.9, and increasing k from 10 to 25 has a factor of approximately 1.3.

5.3.2 Computational breakdown

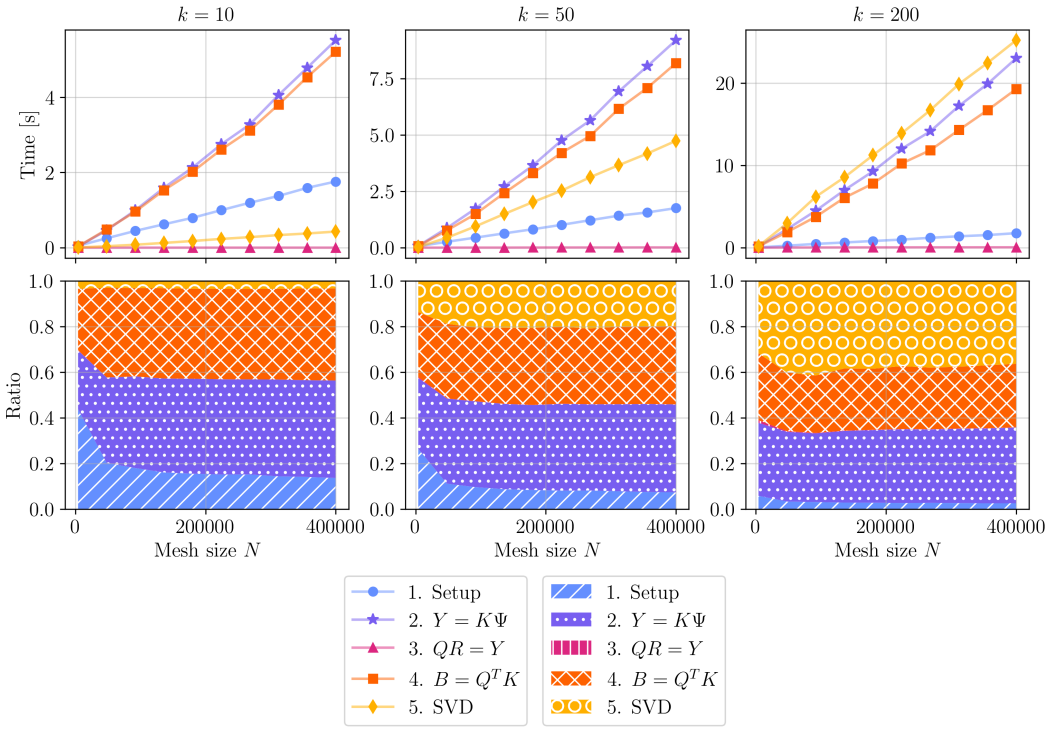


Figure 10: Median computational time per stage in Algorithm 2 for selected N and increasing k. Bottom row shows relative cost. The median is taken over 30 runs for $N \leq 40{,}000$ and 10 runs for $N = 400{,}000$

Figure 10 shows the median computational time for each stage of Algorithm 2 for selected N and increasing k, with the relative computational cost shown in the bottom row. The 'Setup' stage includes factorization of the mass matrix M_∂, and the setup of the K and K^* solvers (Section 3.3). The remaining stages of Algorithm 2 are '2. $Y = K\Psi$' (lines 1–6), '3. $QR = Y$' (lines 7–8), '4. $B = Q^T K$' (lines 9–14), and '5. SVD' (lines 15–18). Table 2 shows relative computational time for selected target ranks and mesh sizes.

The dominant costs of Algorithm 2 are the forward solves using K (stage 2) and the adjoint solves using K^* (stage 4), both of which scale roughly linearly with N, matching the theoretical cost of $\mathcal{O}(lN \log N)$ with $l = k + p$ (Section 3.3). The setup stage has theoretical cost $\mathcal{O}(N^{3/2})$ but appears near-linear in N over the range considered here. Its share of total time decreases with k, because setup is independent of the target rank.

Stage 5's relative cost is primarily determined by the target rank k, but also scales linearly with N, matching the predicted $\mathcal{O}(l^2 N)$ cost of the SVD of B. Its

N	k	Setup	$Y = K\Psi$	$QR = Y$	$B = Q^T K$	SVD
4,000	10	0.44	0.26	0.00	0.27	0.03
	50	0.26	0.31	0.00	0.28	0.14
	200	0.06	0.32	0.01	0.29	0.32
180,000	10	0.16	0.42	0.00	0.39	0.03
	50	0.08	0.37	0.00	0.34	0.21
	200	0.03	0.32	0.00	0.27	0.39
400,000	10	0.14	0.43	0.00	0.40	0.03
	50	0.07	0.39	0.00	0.34	0.20
	200	0.03	0.33	0.00	0.28	0.36

Table 2: Relative computational cost of each stage in Algorithm 2 for selected target ranks k and mesh sizes N.

share grows with k: at $N = 400{,}000$, stage 5 accounts for 3% of total time at $k = 10$ and 36% at $k = 200$. Stage 3 (the QR factorization of the $N_b \times l$ matrix Y) remains below 1% of total time across all mesh sizes.

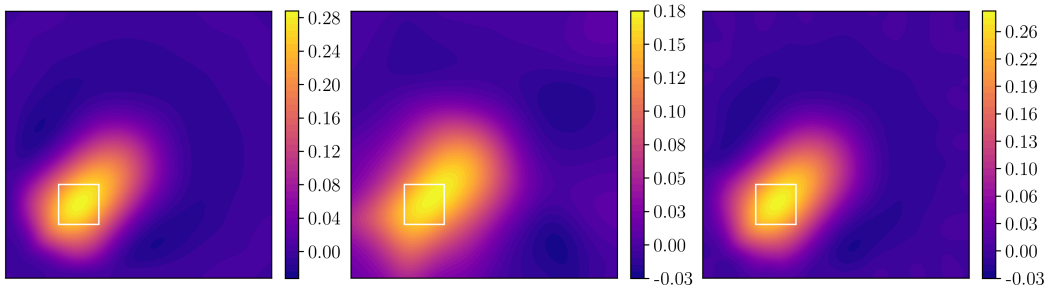
5.3.3 Target rank effect on reconstruction quality

Figure 11 shows the weighted Tikhonov solution of Equation (22), with K and W either exact or approximated by Algorithm 2 at target ranks $k = 10$ and $k = 50$.

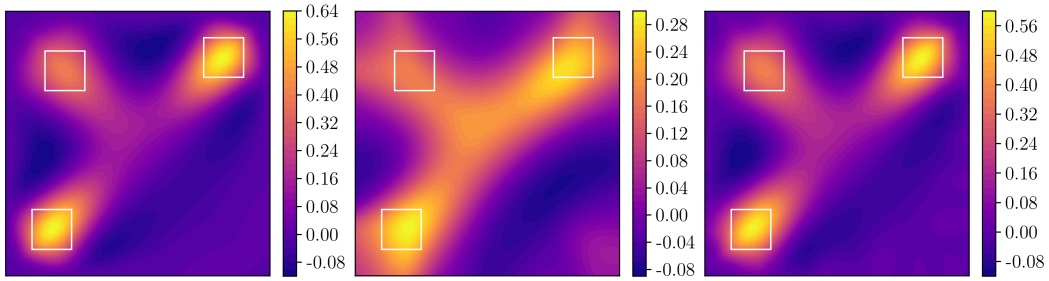
For Problem I (Figure 11a) at $k = 10$, the location of the source is partially identified (middle), slightly shifted towards the interior and with more smoothing than the exact Tikhonov solution (left). The weighted Tikhonov regularization itself already produces significant smoothing [9], and low-rank approximations amplify this effect at small k. At $k = 50$ (right), the reconstruction is visually identical to the exact Tikhonov solution. In Problems II and III (Figures 11b and 11c), $k = 10$ shows the same over-smoothing and shift of each source. These issues disappear at $k = 50$. Across all three problems, $k = 50$ matches the exact Tikhonov reconstruction visually at a small fraction of the computational cost of solving the exact problem (Section 5.3.1).

To measure reconstruction error, we consider three different metrics: Euclidean norm, earth mover's distance (EMD), and the area under the intersection over union curve (AUC-IoU). These metrics emphasize the localization and spatial shape of the reconstructed source, rather than just the amplitude. See Appendix A.7 for details on each metric. An AUC-IoU score of 1 indicates a perfect overlap of the reconstruction and the true source.

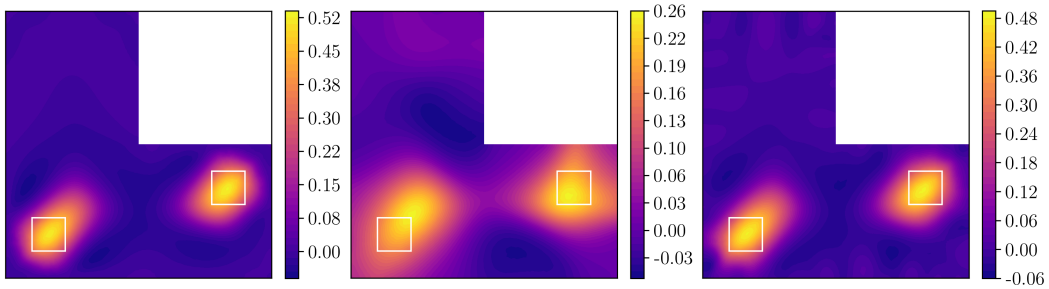
Figure 12 shows the reconstruction error for target ranks $k \in [5, 100]$ and oversampling parameters $p = 0, 5, 10$. The error curves flatten beyond $k \approx 100$, so the figure is cropped there for more detail at smaller k. Table 3 gives the relative



(a) Problem I.

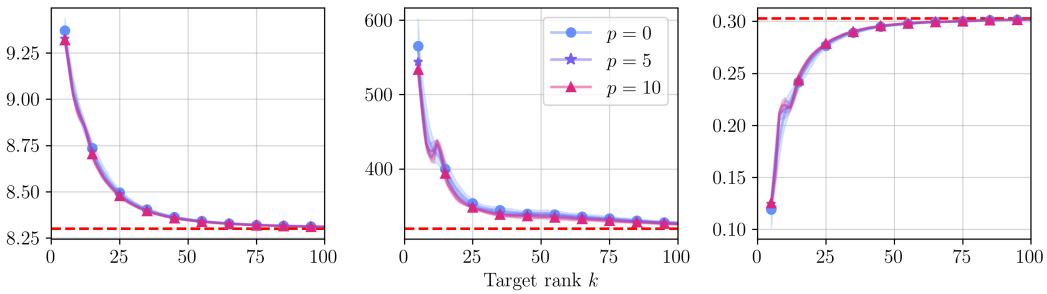


(b) Problem II.

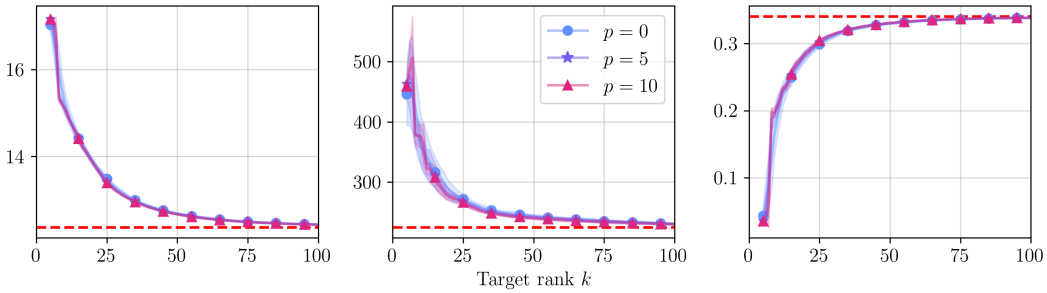


(c) Problem III.

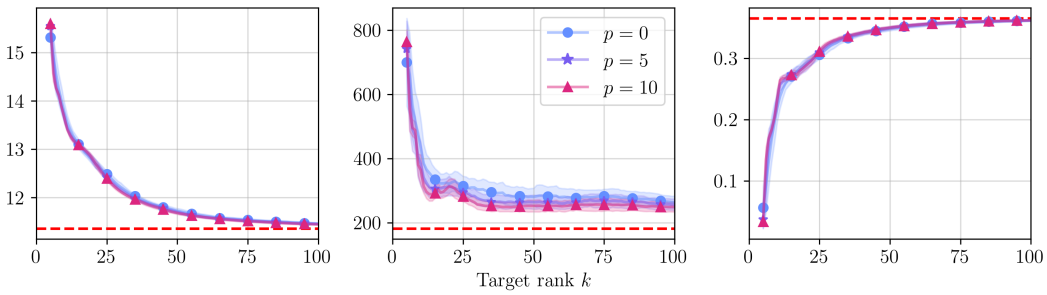
Figure 11: Weighted Tikhonov solutions using exact K and W (left), a rank-10 rSVD approximation of K and W (middle), and a rank-50 rSVD approximation (right), for Problems I, II, and III. True source location outlined in white (Figure 8).



(a) Problem I.



(b) Problem II.



(c) Problem III.

Figure 12: Reconstruction error of weighted Tikhonov solutions using rSVD-approximated K and W (solid lines, mean ± 1 standard deviation over 50 runs) versus the exact weighted Tikhonov solution (dashed line), for a range of target ranks k and oversampling parameters p. Columns: Euclidean (left), EMD (middle), AUC-IoU (right).

	Euclidean (%)			EMD (%)			AUC-IoU (%)		
k	I	II	III	I	II	III	I	II	III
10	7.49	22.40	20.49	32.00	66.57	109.80	27.91	40.53	38.26
25	2.18	8.44	9.34	9.11	18.68	58.86	8.06	10.85	15.40
50	0.56	2.44	2.92	4.74	6.44	39.13	1.97	2.87	4.07
100	0.11	0.50	0.78	1.79	2.03	33.66	0.38	0.59	0.88
199	0.01	0.05	0.15	0.12	0.16	7.85	0.01	0.02	0.04

Table 3: Relative difference (%) between rSVD error and the full-rank exact solution at selected target ranks k, for oversampling $p = 5$. Positive values indicate the rSVD solution is worse than the reference. Mean over 50 runs.

difference (in %) between the rSVD and exact Tikhonov solutions at selected k.

The rSVD reconstruction error approaches the exact reconstruction error as k increases, for all three error metrics. Larger oversampling p yields a small but consistent reduction in error across all three metrics, most visible in the EMD curves. At $k = 50$ and $p = 5$, the Euclidean relative difference is 0.56%, 2.44%, and 2.92% for Problems I, II, and III; the AUC-IoU gives 1.97%, 2.87%, and 4.07%; and the EMD gives 4.74%, 6.44%, and 39.13%. The EMD is more sensitive to low-amplitude noise in the reconstruction, which increases its relative error even though the noise is difficult to see in Figure 11. At $k = 199$ (the largest rank considered), Euclidean and AUC-IoU are within fractions of a percent of the exact Tikhonov values. The EMD ranges from 0.12% to 7.85%, which indicates its sensitivity to small noise or small differences in shape or position compared to the exact Tikhonov solution.

5.3.4 Test vector distribution effect on error

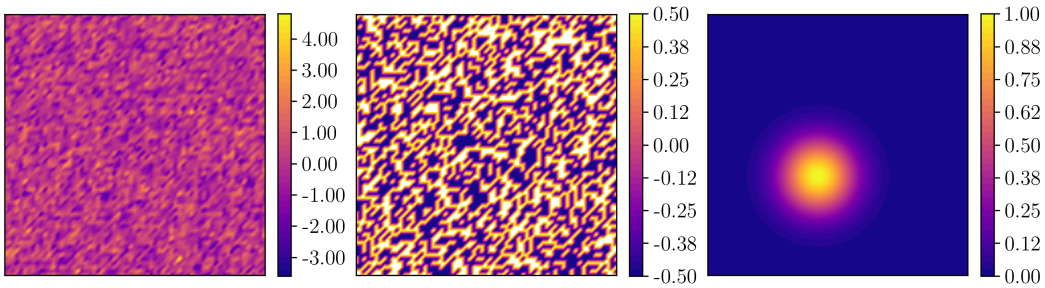


Figure 13: Random test vectors ψ_i drawn from different distributions: Standard Gaussian (left), Rademacher (middle), and Peak (right).

Three test vector distributions are compared: standard Gaussian ($\psi_{ij} \sim \mathcal{N}(0, 1)$), Rademacher ($\psi_{ij} \in \{-\frac{1}{2}, \frac{1}{2}\}$ with equal probability, see [51]), and a spatially localized

Gaussian peak, which we refer to as a Peak test vector:

$$\psi_i(x) = \exp\left(-\frac{|x - x_0|^2}{2\sigma_p^2}\right),$$

where the center of the peak x_0 is sampled uniformly from the domain and $\sigma_p = 0.1$. Examples are shown in Figure 13.

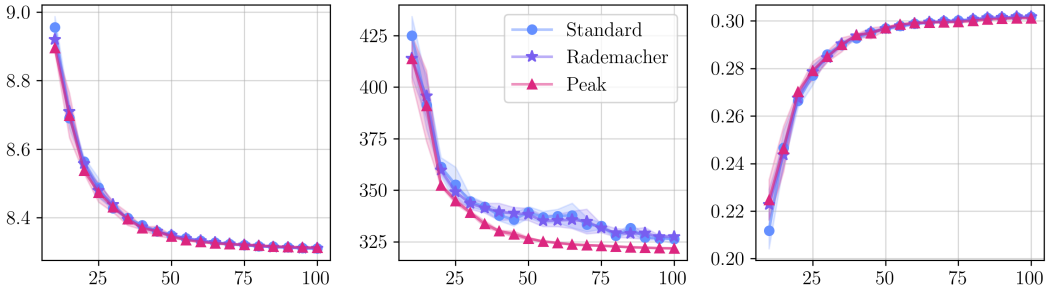


Figure 14: Comparison of reconstruction error using rSVD (Algorithm 2) with sampling vectors ψ_i from different distributions (using Problem I). Error measured in: Euclidean norm (left), EMD (middle), and AUC-IoU (right). Mean ± 1 standard deviation over 50 runs.

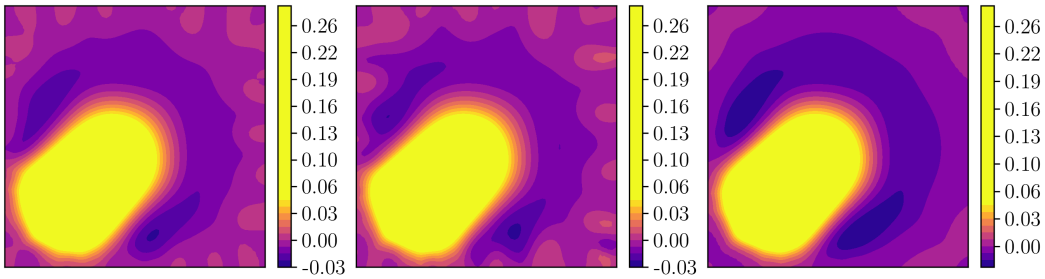


Figure 15: Reconstructions using different sampling distributions (ψ_i) in the rSVD approximation (Algorithm 2, Problem I, $k = 50$): Standard Gaussian (left), Rademacher (middle), and Peak (right). Colormap clipped to reveal low-amplitude noise.

Figure 14 compares reconstruction error across the three distributions for Problem I (similar results hold for Problems II and III). Rademacher and standard Gaussian test vectors yield nearly identical errors across all three metrics. The Peak test vectors produce lower EMD error and smaller run-to-run variance. Figure 15 shows individual reconstructions with the colormap clipped to 0.05, revealing that the Peak test vectors generate less low-amplitude noise than Gaussian or Rademacher.

5.3.5 Mesh resolution effect on reconstruction error

In Figure 16, we see that fixing the target rank k and refining the mesh resolution N does not decrease the Euclidean or AUC-IoU error. The EMD shows a slight

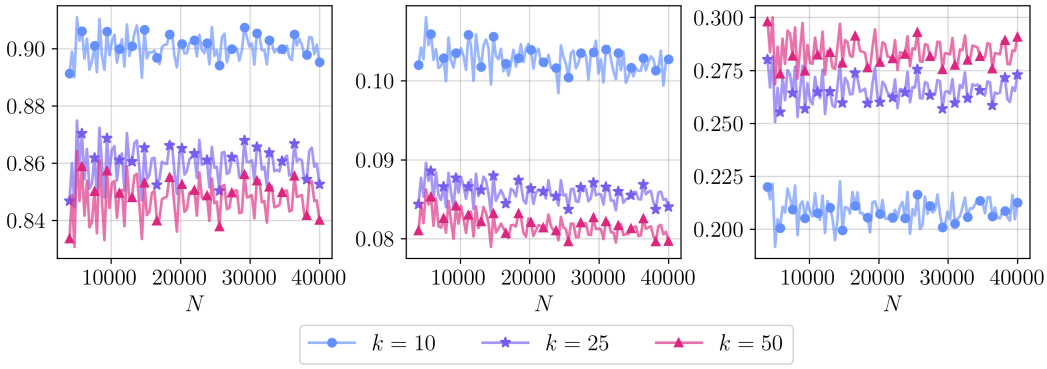


Figure 16: The reconstruction error using rSVD with fixed k and increasing mesh resolution N. Error measured using relative Euclidean norm (left), relative EMD (middle), and AUC-IoU (right) on Problem I. Values averaged over 5 repetitions.

decrease with N, likely because finer meshes produce smoother spatial transitions between neighboring mesh elements.

Although $\text{rank}(K) = N_b$ grows with N, the target rank k need not be adjusted: refining the mesh adds no new information about the source f.

5.3.6 Approximation of singular values

p	rSVD on	$\tilde{\sigma}_{10}/\sigma_{10}$	$\tilde{\sigma}_{25}/\sigma_{25}$	$\tilde{\sigma}_{50}/\sigma_{50}$	$\tilde{\sigma}_{100}/\sigma_{100}$
0	K	1.000	0.993	0.962	0.566
	K^*	0.999	0.977	0.919	0.503
5	K	1.000	0.995	0.969	0.668
	K^*	0.999	0.983	0.929	0.595
10	K	1.000	0.995	0.973	0.724
	K^*	0.999	0.986	0.939	0.642

Table 4: Ratio $\tilde{\sigma}_i/\sigma_i$ for selected singular values, across oversampling parameters $p = 0$, 5, 10 using rSVD on K (Algorithm 2) and rSVD on K^* (Algorithm 3). Mean over 20 runs using Problem I.

Table 4 shows that both rSVD on K and rSVD on K^* underestimate the true singular values σ_i, with the bias increasing for higher-index values. Larger p reduces the underestimation, and rSVD on K^* underestimates more than rSVD on K across all tested p.

At $k = 50$, reconstruction errors were within 1%-4% of the exact Tikhonov solution in terms of Euclidean and AUC-IoU (Section 5.3.3). At this rank with $p = 5$, the ratio $\tilde{\sigma}_{50}/\sigma_{50}$ is 0.969 for rSVD on K and 0.929 for rSVD on K^*: the underestimation is small where reconstruction quality is already high. If $\tilde{\sigma}_i \ll \sigma_i$, the approximated

forward operator is more ill-posed than the true one, and reconstruction quality degrades. This is not the case at $k = 50$, where $\tilde{\sigma}_i/\sigma_i$ remains close to 1.

5.3.7 Computational time of adjoint operator

	$k = 10$		$k = 50$		$k = 200$	
N	t_1/t	t_2/t	t_1/t	t_2/t	t_1/t	t_2/t
4000	1.29	1.30	1.22	1.31	1.12	1.31
28000	1.25	1.26	1.24	1.32	1.09	1.31
52000	1.31	1.33	1.24	1.35	1.08	1.30
76000	1.29	1.30	1.24	1.36	1.09	1.32
100000	1.29	1.30	1.22	1.35	1.09	1.32

Table 5: Relative computational time of computing rSVD on K^* compared to rSVD on K (time t) for selected k and N. Two relative times are reported: t_1/t, where t_1 is the time for rSVD on K^* alone (Algorithm 3), and t_2/t, where t_2 includes the recovery of the SVD of K from the SVD of K^* (Section 3.2.1). Values are medians over 50 runs.

Table 5 shows that rSVD on K^* takes approximately 1.3 times longer than rSVD on K for $k = 10$, 1.2 times longer for $k = 50$, and 1.1 times longer for $k = 200$, across all reported N. When the recovery of the SVD is included (t_2/t), it takes approximately 1.3 times as long across all k and N.

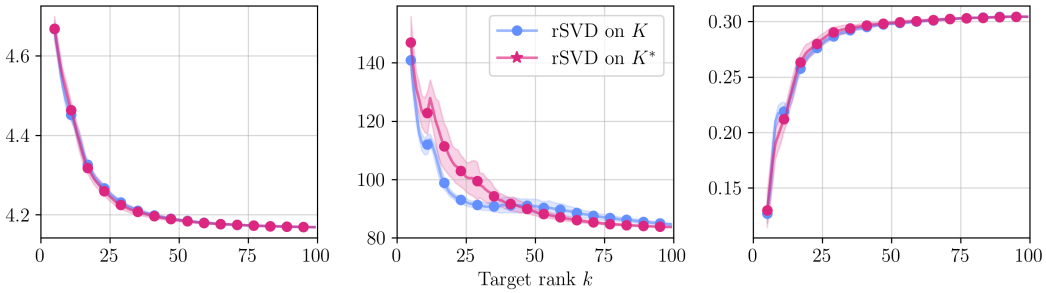
For larger k, the recovery of the SVD takes up an increasing amount of time, as it involves QR factorizations of an $N_b \times k$ and an $N \times k$ matrix, in addition to an SVD of a $k \times k$ matrix (Section 3.2.1). Additionally, as k goes to 200, the ratio t_1/t gets closer to 1.0, as an increasing amount of the time of Algorithm 2 goes to the SVD step (Section 5.3.2), which for Algorithm 3 is much cheaper (SVD of $l \times N$ versus $l \times N_b$ matrix).

5.3.8 Reconstruction error using the adjoint operator

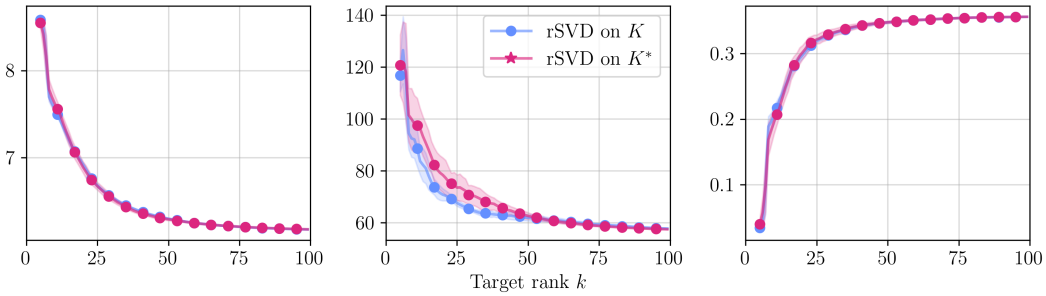
Figures 17 and 18 and Table 6 compare reconstruction error between rSVD on K and rSVD on K^* over the range $k \in [10, 100)$.

In terms of Euclidean norm, both methods give practically the same error, within $\pm 1\%$ across all problems and target ranks. For AUC-IoU, the relative difference at $k = 10$ is between -6% and -4%, and between -2% and 1% for $k \geq 25$ with no consistent direction.

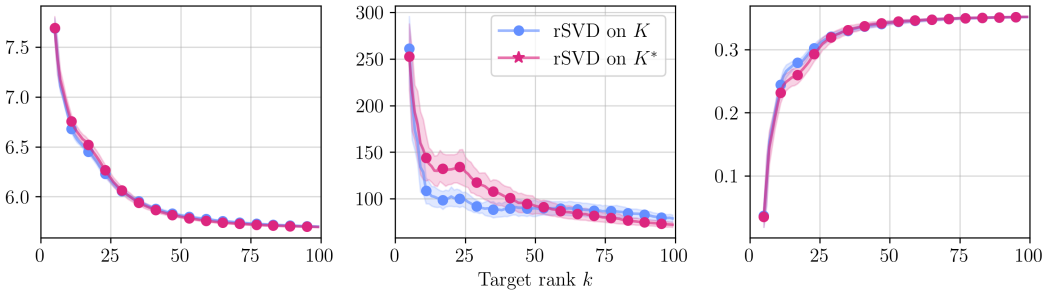
The EMD shows that at $k = 10$, rSVD on K gives 10% to 19% worse reconstructions than rSVD on K^*, though at this rank the smoothing and positional shift of the reconstruction are already significant (Section 5.3.3). In the range $k \approx 20$ to $k \approx 50$, rSVD on K^* performs worse: at $k = 25$, Algorithm 2 performs 9% to 31%



(a) Problem I.



(b) Problem II.

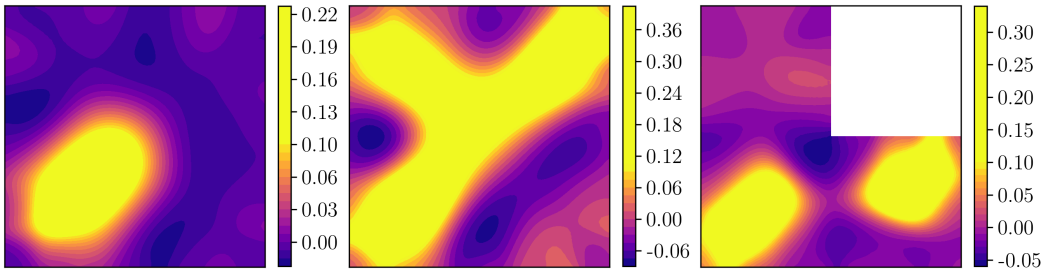


(c) Problem III.

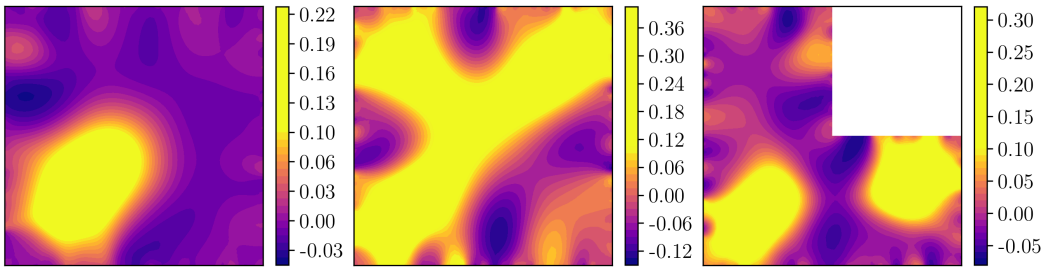
Figure 17: Reconstruction error using rSVD on K (Algorithm 2) versus rSVD on K^* (Algorithm 3) for increasing k. Mean ± 1 standard deviation over 50 runs. Columns: Euclidean (left), EMD (middle), AUC-IoU (right).

	Euclidean (%)			EMD (%)			AUC-IoU (%)		
k	I	II	III	I	II	III	I	II	III
10	-0.50	-0.76	-0.80	-9.66	-6.52	-19.01	-6.48	-4.69	-3.52
25	-0.17	-0.13	0.39	9.27	8.75	30.63	-1.36	-0.63	2.05
50	0.02	0.11	0.22	2.10	-1.11	-2.74	0.24	0.02	0.07
75	0.00	0.02	-0.18	-2.41	-0.86	-7.94	0.01	-0.00	-0.04
99	-0.01	-0.03	0.07	1.10	0.51	8.57	-0.00	-0.00	0.04

Table 6: The relative error difference by using rSVD on K^* (Algorithm 3) compared to rSVD on K (Algorithm 2) for Problems I, II, and III. A positive number means rSVD on K performs better. Mean over 50 runs.



(a) rSVD on K (Algorithm 2)



(b) rSVD on K^* (Algorithm 3)

Figure 18: Weighted Tikhonov solutions with $k = 15$ for Problem I (left), II (middle), and III (right), using Algorithm 2 (top) and Algorithm 3 (bottom). The colormap is clipped at 0.1 to expose low-amplitude noise in the reconstruction.

better across all three problems. The difference is a result of noise: reconstructions from rSVD on K^* contain more low-amplitude noise than those from rSVD on K (Figures 18b and 18a), to which the EMD is particularly sensitive.

5.3.9 Accuracy on noisy data

We measure the reconstruction error for the transformed weighted Tikhonov problem of Equation (24). We replace $y = Kx$ with $\tilde{y} = y + \epsilon$, where $\epsilon_i \sim \mathcal{N}(0, 1)$ and $\frac{\|\epsilon\|}{\|y\|} = \delta$ with $\delta = 0.01$. The regularization parameter is set to $\lambda = \lambda_{\text{dp}}$, found using the discrepancy principle with safety factor $\nu_{\text{dp}} = 10$ (see Section 5.2 in Hansen [8]), so that λ is selected independently for each method and rank.

Problem	Method	Euclidean	EMD	AUC-IoU
I	rSVD	60	20	60
	tSVD	60	20	80
II	rSVD	50	20	65
	tSVD	45	20	65
III	rSVD	75	30	$\geq$100
	tSVD	85	30	95

Table 7: The optimal target/truncation rank k^* of rSVD/tSVD on noisy data using Problems I, II, and III in terms of error metrics (mean over 50 runs): Euclidean norm, EMD, and AUC-IoU.

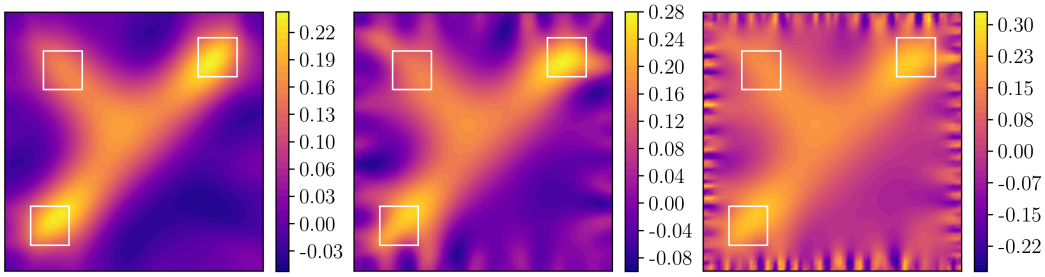
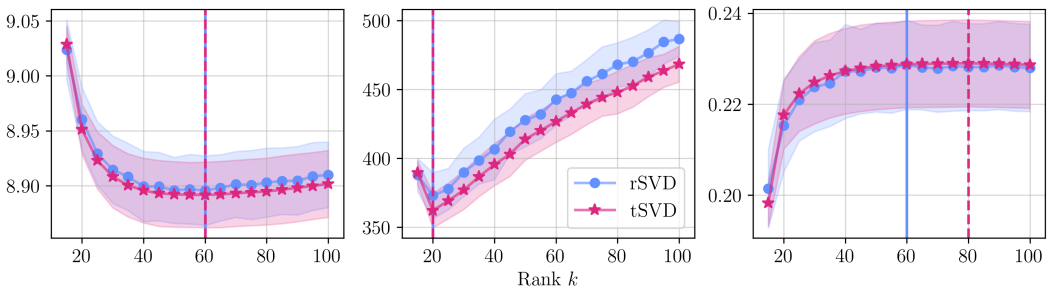


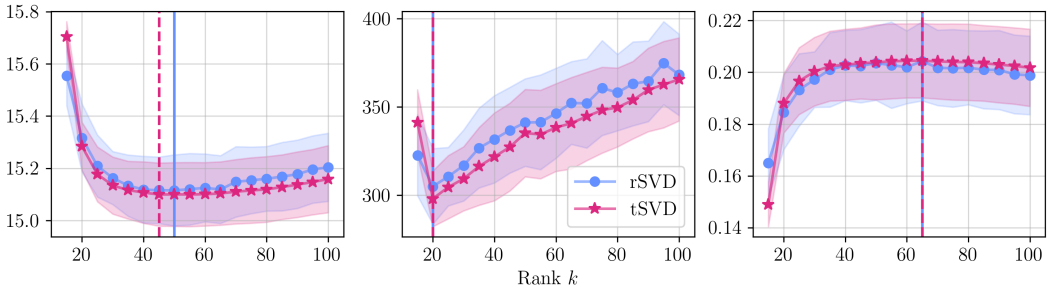
Figure 19: Reconstructions (Problem II) on a noisy problem using rSVD and $\lambda = \lambda_{\text{dp}}$ for target ranks 20 (left), 60 (middle), and 150 (right). True source locations outlined in white.

Figure 20 and Table 7 report reconstruction error and optimal ranks for rSVD (Algorithm 2) and tSVD (Section 2.4.3) on the noisy problem. Figure 19 illustrates how a target rank that is too high includes noise in the reconstruction.

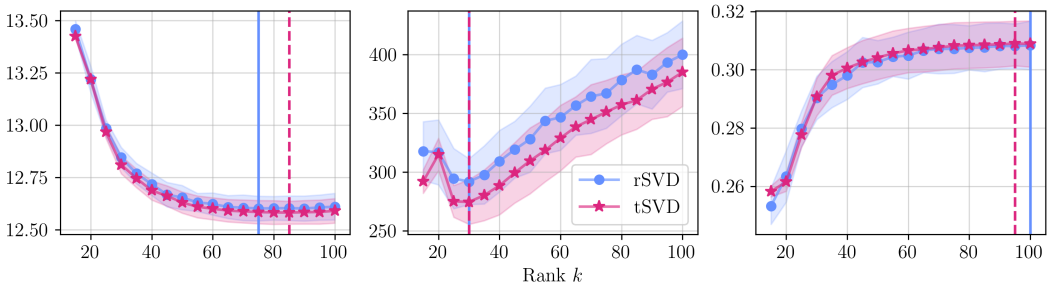
In terms of Euclidean error, the optimal rank is between 50 and 75 for rSVD, and 45 and 85 for tSVD. The EMD is more sensitive to higher ranks which include more noise (Figure 19), and the optimal rank for EMD is between 20 and 30 for



(a) Problem I.



(b) Problem II.



(c) Problem III.

Figure 20: Reconstruction error using rSVD (Algorithm 2) and tSVD to approximate K and W at different ranks on noisy problems. Error measured in: Euclidean norm (left), EMD (middle), AUC-IoU (right). Average over 50 runs with ± 1 standard deviation. Optimal rank marked with vertical solid and dashed lines for rSVD and tSVD, respectively.

both methods. For AUC-IoU, optimal ranks are between 60 and 65 for rSVD, and between 65 and 95 for tSVD, except for Problem III with rSVD where the optimal rank exceeds the tested range.

At their respective optimal ranks, rSVD and tSVD give nearly identical Euclidean errors and AUC-IoU scores across the three problems, with differences below 0.25%. The largest gap is in EMD: tSVD achieves 2.35% to 6.33% lower EMD, with the largest difference on Problem III.

5.4 Dynamical low-rank approximation

Unless stated otherwise, all experiments use a rank-50 rSVD approximation of K and W (Algorithm 2, $p = 5$, standard Gaussian test vectors ψ_i) to form Φ, its gradient, and its Hessian (Section 4.3), on which Algorithm 5 is applied to find a low-rank solution.

We write $X^* \in \mathbb{R}^{n\times n}$ for the matrix form of the true source, and N_r for the restart period of restarted DLRA-CG (Section 4.4.1). Convergence is measured by the relative reconstruction error

$$e^{(i)} = \frac{\|X^{(i)} - X^*\|_F}{\|X^{(0)} - X^*\|_F},$$

which equals 1 at initialization and decreases toward 0 as the iterates approach X^*. Unless stated otherwise, $X^{(0)}$ has entries drawn uniformly from $[0, 10^{-3}]$.

5.4.1 Rate of convergence

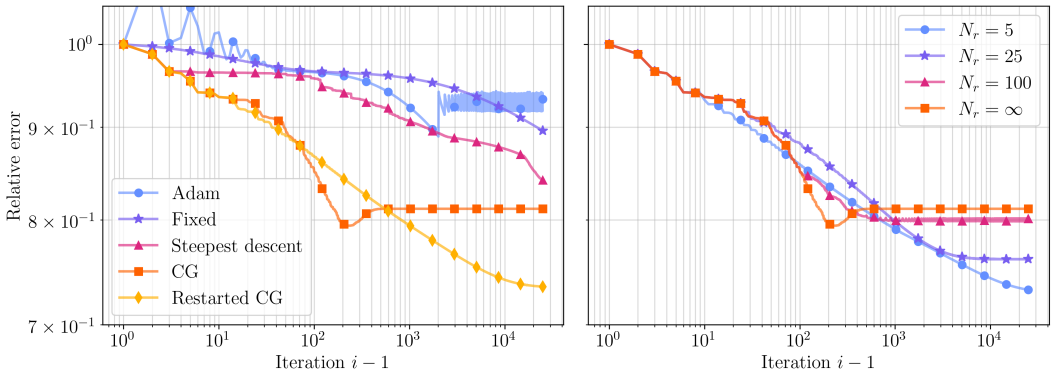


Figure 21: Relative reconstruction error $e^{(i)}$ versus iteration count. Left: three DLRA step-size rules (Adam, fixed step, steepest descent) and DLRA-CG (with and without restarting, using $N_r = 10$ for restarting). Right: DLRA using CG with different restart periods N_r, with $N_r = \infty$ being no restarting.

Figure 21 compares the reconstruction error per iteration for three DLRA step-size rules (Algorithm 4) and DLRA-CG (Algorithm 5), with and without restarting.

Method	< 0.90	< 0.85	< 0.80	< 0.75
Adam	1665	–	–	–
Fixed	21751	–	–	–
Steepest descent	1449	20916	–	–
DLRA-CG	48	100	184	–
Restarted DLRA-CG	35	165	816	5941

Table 8: Number of iterations to reach a given relative reconstruction error $e^{(i)}$ for each method (Problem I), where – indicates the threshold was not reached. For restarted CG, $N_r = 10$ was used.

The right panel shows the effect of the restart period N_r on the convergence of DLRA-CG.

DLRA-CG reaches the lowest error fastest but then the error rises, showing a form of semi-convergence, as the full-rank trajectory $Y^{(i)}$ drifts away from $X^{(i)}$ (Section 4.4.1). Restarted DLRA-CG (with $N_r = 10$) avoids this by periodically correcting the CG recurrence to $X^{(i)}$, at the cost of slower convergence, but converges faster than the three DLRA step-size rules, which all fail to converge within 25,000 iterations (and the Adam rule gets stuck).

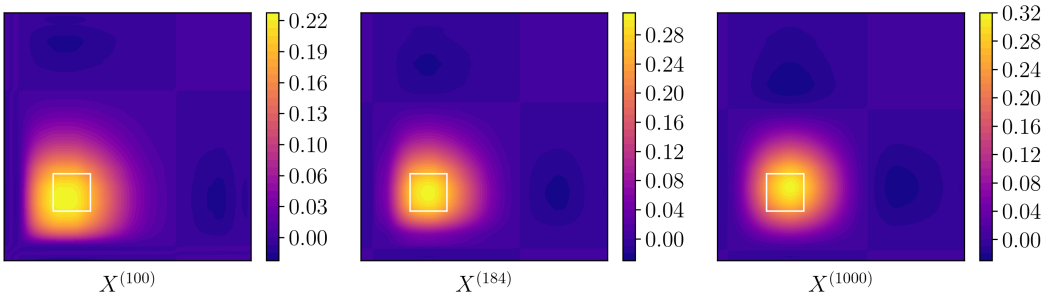
A smaller N_r slows convergence but drives the error lower, and for larger N_r, the semi-convergent behavior of DLRA-CG re-appears. Table 8 shows the iteration count to reach selected error thresholds. DLRA-CG reaches a relative error of 0.9 in 48 iterations, and restarted DLRA-CG takes 35 iterations. The step-size rules Adam, fixed, and steepest descent reach this threshold in 1665, 21751, and 1449 iterations. To reach a threshold of 0.85, DLRA-CG takes 100 iterations and restarted DLRA-CG takes 165. At 0.75, only restarted DLRA-CG succeeds, requiring 5941 iterations.

Per-iteration compute times are similar across all methods (0.5-0.7 ms), so the iteration counts in Table 8 translate directly to runtime. At $e^{(i)} < 0.85$, DLRA-CG is approximately 210 times faster than steepest descent.

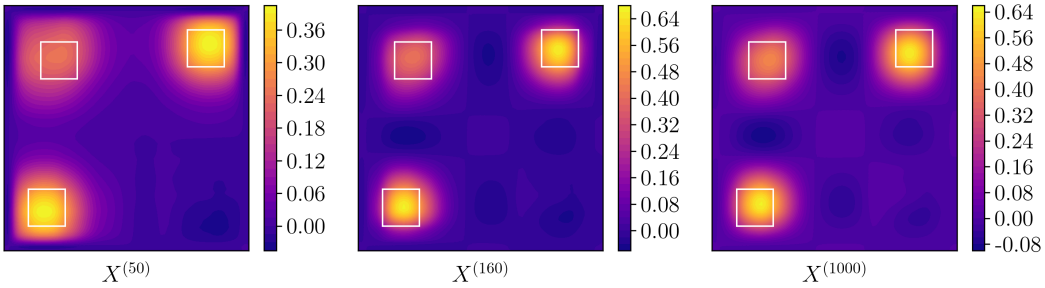
5.4.2 Quality of the low-rank solutions

DLRA-CG shows semi-convergence: the iterates initially approach the true source, but start to move away from the optimal solution when $X^{(i)}$ and $Y^{(i)}$ drift apart (Section 4.4.1). Figure 22 shows the difference between the minimum reconstruction iterate and the converged iterate. In both Problems I and II, the converged iterate (right panels) shows a slight inward shift of the source positions, while the minimum-error iterate (middle panels) correctly reproduces the source locations. The early iterate (left panels) has not yet localized the sources.

Compared to the full-rank solutions of Section 5.3.3, which have smoothing pulled toward the interior, the low-rank solutions are more uniformly smoothed with no such tail. For Problem I, this creates a circular source, and for Problem II, this results in cleanly separated sources (in Figure 11b they were connected). Additionally, at



(a) Problem I and $r = 1$. $e^{(i)}$ from left to right: 0.85, 0.80, 0.81.



(b) Problem II and $r = 2$. $e^{(i)}$ from left to right: 0.78, 0.65, 0.67.

Figure 22: Iterates $X^{(i)}$ of Algorithm 5 (no restarting) at three stages of the same run: an early iterate before the error minimum (left), the iterate at minimum reconstruction error (middle), and a converged iterate at which the error has risen above the minimum due to semi-convergence (right). True source outlined in white.

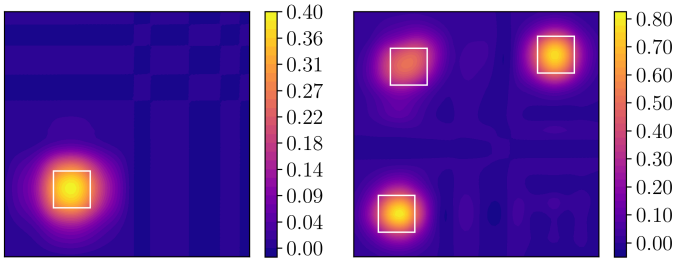
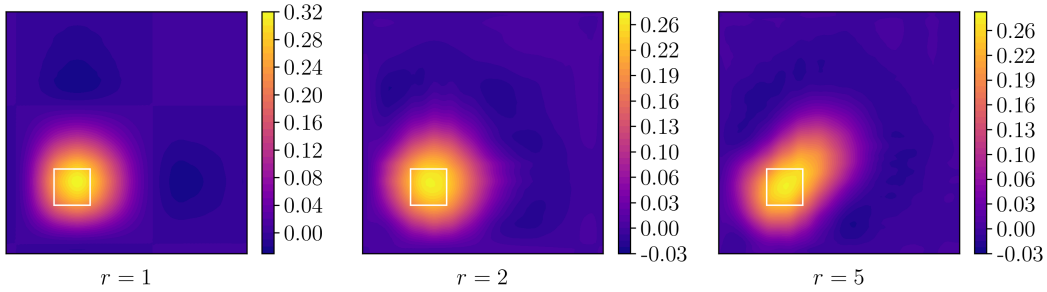


Figure 23: Reconstructions using restarted CG ($N_r = 10$) with 25,000 iterations (run until convergence), with same respective $X^{(0)}$s as in Figure 22. Left: Problem I, rank 1, $e^{(i)} = 0.73$. Right: Problem II, rank 2, $e^{(i)} = 0.63$. True source outlined in white.

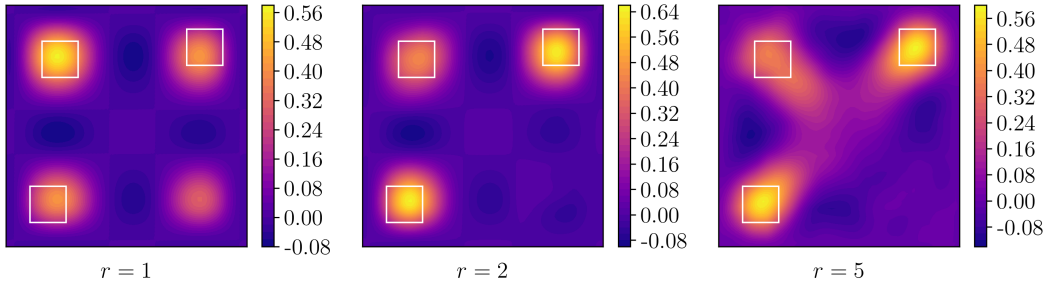
$k = 50$, the peak amplitudes are 0.28 (rank-1 DLRA-CG) versus 0.26 (full-rank rSVD) for Problem I, and 0.64 versus 0.48 for Problem II, where the upper-left source is weaker in the rank-2 solution. The true source amplitude is 1.0 (Figure 8). The low-amplitude noise in the full-rank rSVD solution is replaced by a checker-like pattern of similar amplitude.

Restarted DLRA-CG ($N_r = 10$) achieves a lower relative error than DLRA-CG without restarting (Figure 21, left). The converged solutions in Figure 23 are correctly located and less diffuse, with peak amplitudes of 0.40 and 0.80 for Problems I and II, closer to the true amplitude of 1.0 than both the full-rank rSVD Tikhonov solutions and the solutions without restarting.

5.4.3 Effect of rank on reconstructions



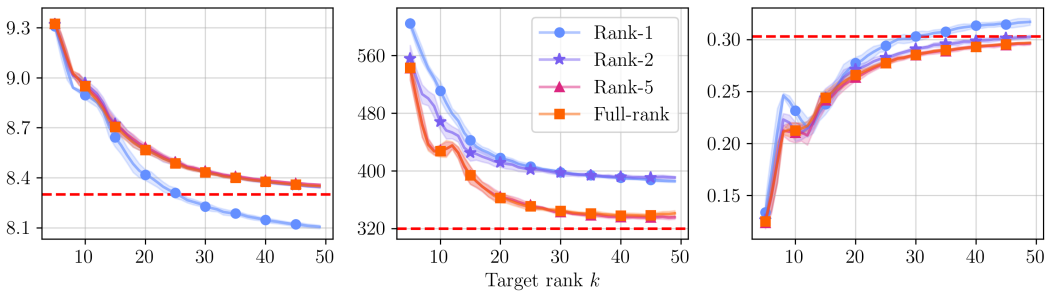
(a) Problem I: The rank-1 solution (left) gives the best reconstruction. $e^{(i)}$ from left to right: 0.81, 0.83, 0.84.



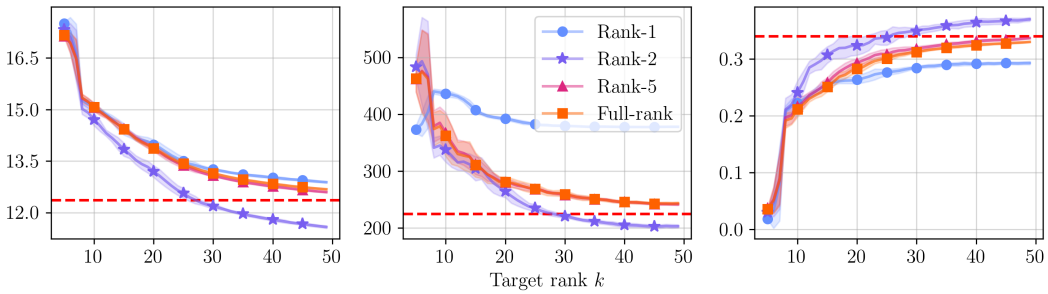
(b) Problem II: The rank-1 solution (left) places an additional source at bottom right, where no true source exists; the rank-2 solution gives the best reconstruction (middle). $e^{(i)}$ from left to right: 0.75, 0.67, 0.73.

Figure 24: Low-rank solutions of Algorithm 5 (no restarting) at selected ranks $r = 1$ (left), $r = 2$ (middle), and $r = 5$ (right). True source outlined in white.

Figure 24 shows DLRA-CG solutions for Problems I and II at ranks $r = 1, 2, 5$. For Problem I, the reconstruction error increases with rank, and at rank 5 the solution resembles the full-rank Tikhonov reconstruction (Figure 11a). For Problem II, rank 2 is optimal, even though the true source is of rank 3 (reported in Section 5.4.4; discussed in Section 6.2.3). At rank 5 the reconstruction resembles the



(a) Problem I.



(b) Problem II.

Figure 25: Reconstruction error vs. rSVD target rank k: full-rank Tikhonov solutions (rank-k rSVD approximation of K and W, Algorithm 2) and DLRA-CG solutions at ranks $r = 1, 2, 5$ using the same approximation. Exact Tikhonov solution: dashed line. Columns: Euclidean norm (left), EMD (middle), AUC-IoU (right). Mean ± 1 standard deviation over 30 runs.

full-rank case (Figure 11b).

Figure 25 shows that for Problem I, the rank-1 DLRA-CG solution achieves lower Euclidean and AUC-IoU error than the full rank rSVD solution. For EMD, the full-rank solution is better, possibly because the focused distribution of the rank-1 solution differs more from the flat true source than the smoother full-rank reconstruction. For Problem II, the rank-2 solution outperforms rank-1, rank-5, and full-rank solutions across all metrics for $k > 10$. Reconstructions of rank 5 achieve approximately the same error as full-rank rSVD reconstructions.

As shown in Section 5.3.3, full-rank rSVD solutions approach, but do not outperform the exact Tikhonov solution in the range $k \in [5, 100]$ (Figure 12b). The low-rank solutions do: for Problem I, the rank-1 solution surpasses the exact Tikhonov solution in Euclidean error from $k = 26$ and in AUC-IoU from $k = 31$, though not in EMD. For Problem II, the rank-2 solution surpasses it across all metrics by $k = 28$.

5.4.4 Residuals of low-rank solutions

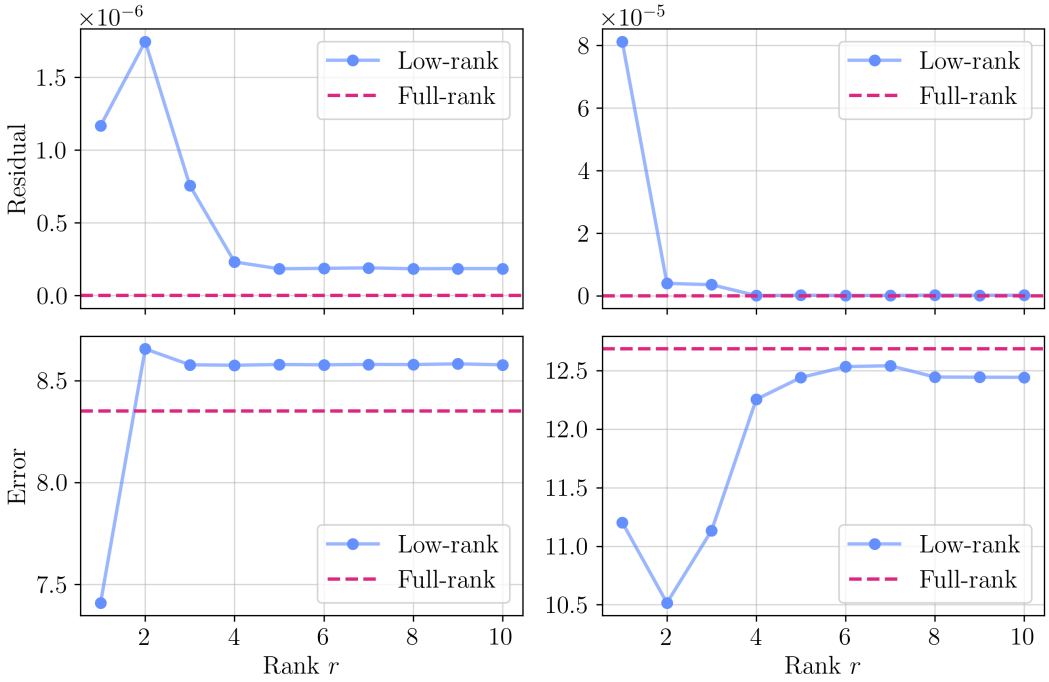


Figure 26: The residuals (top row) and Euclidean error (bottom row) of low-rank solutions for ranks $r = 1, 2, \ldots, 10$ using Problem I (left) and II (right). Dashed line: residual/error of the full-rank solution. Solutions computed using Algorithm 5 with $N_r = 10$.

For Problem I, a rank-1 solution gives the best error, and for Problem II, a rank-2 solution (Figure 26). The Problem I residual rises and then falls with increasing rank, with no obvious connection to the optimal rank, while the Problem II residual shows a sharp corner at the transition from rank 1 to rank 2. Using the residual

curves could serve as a method for deciding a good rank, in particular to avoid choosing r too small.

5.4.5 Rate of convergence using preconditioners

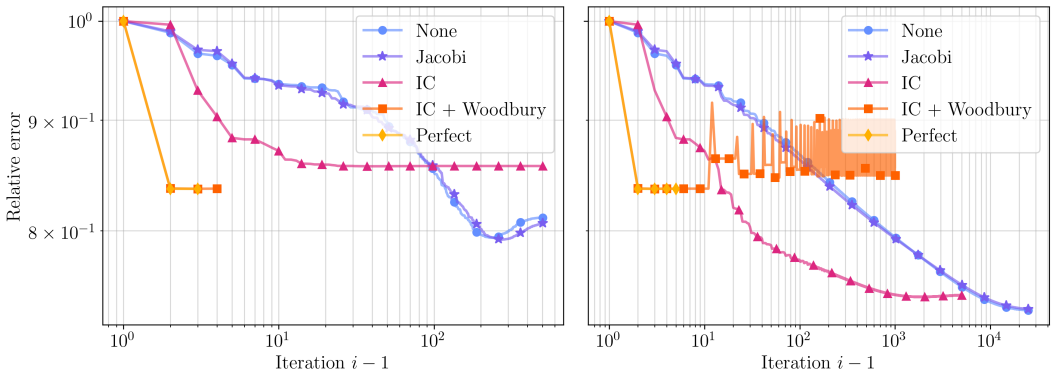


Figure 27: The relative reconstruction error $e^{(i)}$ versus iteration count using DLRA-PCG (Algorithm 6) with four different preconditioners (Jacobi, IC, IC + Woodbury, Perfect) and no preconditioning (DLRA-CG). Left: no restarting. Right: restarting with $N_r = 10$. Problem I was used.

Preconditioner	N_r	< 0.90	< 0.85	< 0.80	< 0.75
None	∞	48	100	184	-
	25	54	249	958	-
	10	35	165	816	5941
Jacobi	∞	43	112	204	-
	25	50	250	1042	-
	10	34	145	776	6518
IC	∞	4	-	-	-
	25	4	28	57	-
	10	4	14	30	680
IC+Woodbury	∞	1	1	-	-
Perfect	∞	1	1	-	-

Table 9: Number of iterations to reach a given relative reconstruction error $e^{(i)}$ using DLRA-PCG with different preconditioners and restart periods N_r (Problem I), where - indicates the threshold was not reached.

The left panel of Figure 27 shows that, except for Jacobi, using preconditioners speeds up the convergence significantly at the cost of higher error. IC, IC + Woodbury, and a perfect preconditioner show faster convergence compared to no preconditioning, but the final error is higher: 0.86, 0.84 and 0.84 relative error, compared to 0.81 for Jacobi and no preconditioning.

The right panel shows DLRA-PCG combined with restarting. For Jacobi, this gives a similar effect as restarted DLRA-CG: slower convergence but better final error. For the IC preconditioner, restarting reduces the final error steadily. The IC + Woodbury preconditioner shows worse results when combined with restarting: the error rises and the iterates diverge. Restarting has no effect on the perfect preconditioner.

The Jacobi, IC, and IC+Woodbury preconditioners have setup times of approximately 2, 4, and 11 ms and per-iteration costs of 0.75, 0.80, and 1.17 ms, compared to 0.74 ms without preconditioning. IC with $N_r = 10$ reaches $e^{(i)} < 0.80$ and < 0.75 fastest among all tested combinations (Table 9): 9 times fewer iterations and 8 times faster in computational time than without preconditioning. IC+Woodbury and Perfect reach $e^{(i)} < 0.85$ in a single iteration but stall there.

5.4.6 Local minima

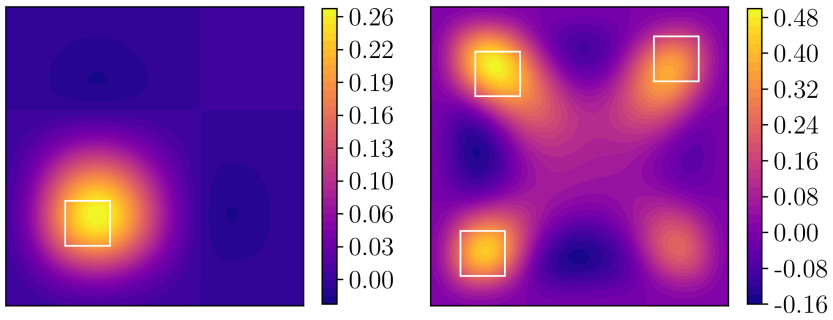


Figure 28: Suboptimal solutions using DLRA-PCG with IC + Woodbury preconditioner and $N_r = 10$. True sources outlined in white. Left: Problem I, $r = 1$, $e^{(i)} = 0.85$. Right: Problem II, $r = 2$, $e^{(i)} = 0.76$.

From Figure 27 we saw that using DLRA-PCG with IC + Woodbury preconditioning and $N_r = 10$, the iterates diverged. Figure 28 shows these solutions after 500 iterations. For Problem I, the reconstruction is slightly shifted, with $e^{(i)} = 0.85$. For Problem II, an incorrect source is seen in the lower right, and the relative error is $e^{(i)} = 0.76$.

For Problem II, when using the suboptimal solution from the left panel of Figure 28 as the initial iterate $X^{(0)}$ for DLRA-CG and DLRA-PCG, the solvers were not able to escape this suboptimal solution (except when using a perfect preconditioner). This suggests that this point is a local minimum on the manifold of rank-2 matrices. Restarting resolved the problem. This issue was not observed for Problem I.

6 Discussion

6.1 Matrix-free rSVD

6.1.1 Asymptotic analysis

To explain the computational times and asymptotic behavior in Sections 5.3.1 and 5.3.2, we examine the cost of forming K and of Algorithm 2 in more detail. Standard dense SVD and QR costs throughout this and the following section follow [22].

For a 2D elliptic problem, the Cholesky factorization of A_h and M_∂ can be done in $\mathcal{O}(N^{3/2})$ and $\mathcal{O}(N_b^{3/2})$ computations, respectively (Section 3.3). The resulting triangular systems can be solved at the respective costs $\mathcal{O}(N \log N)$ and $\mathcal{O}(N_b \log N_b)$. Since $N_b \ll N$, we may disregard the factorization and triangular solves using M_∂.

The cost of computing $K = TA_h^{-1}M$ involves solving N_b linear systems $A_h v_i = T^T e_i$ (Section 3), each system costing $\mathcal{O}(N \log N)$ using a sparse Cholesky factorization of A_h. With $N_b \sim \sqrt{N}$, the cost therefore scales as $\mathcal{O}(N^{3/2} \log N)$, explaining the scaling of t_K observed in the left panel of Figure 9. The SVD of dense matrix $K \in \mathbb{R}^{N_b \times N}$ costs $\mathcal{O}(N_b^2 N) = \mathcal{O}(N^2)$, which is larger than $\mathcal{O}(N^{3/2} \log N)$, explaining t_{SVD} in Figure 9.

For the matrix-free rSVD (Algorithm 2), factorizing A_h costs $\mathcal{O}(N^{3/2})$, and is done once. Each application of K and K^* costs $\mathcal{O}(N \log N)$, totaling $\mathcal{O}(lN \log N)$. The SVD of the $l \times N$ matrix B costs $\mathcal{O}(l^2 N)$, and the cost of the QR factorization of the small $N_b \times l$ matrix Y is $\mathcal{O}(l^2 N_b)$, and may be disregarded. Hence, assuming two dimensions and an efficient sparse Cholesky factorization of A_h, Algorithm 2 costs

$$\underbrace{\mathcal{O}(N^{3/2})}_{\text{setup}} + \underbrace{\mathcal{O}(lN \log N)}_{\substack{\text{forward and} \\ \text{adjoint solves}}} + \underbrace{\mathcal{O}(l^2 N)}_{\text{SVD of } B}. \tag{30}$$

This is consistent with the results in the right panel of Figure 9: t_{rSVD} scales near-linearly on the observed range, matching the near-linear scaling of $\mathcal{O}(N \log N)$. Within the observed range, the $\mathcal{O}(lN \log N)$ solve cost dominates the one-time $\mathcal{O}(N^{3/2})$ setup (Figure 10), and $N \log N$ is close enough to linear that t_{rSVD} in Figure 9 appears near-linear.

Equation (30) also explains the cost breakdown in Figure 10. For larger k, a larger share is taken up by the SVD stage, as predicted by $\mathcal{O}(l^2 N)$. For $k = 200$ ($l = k+p$), the largest considered target rank, we observed that as N increases, the forward and adjoint solves start taking up a larger share of the time compared to the SVD of B, also consistent with Equation (30). The cost $\mathcal{O}(l^2 N_b)$ of the QR-factorization $QR = Y$ is negligible, matching the results for all k and N.

6.1.2 Overhead of the adjoint formulation

The adjoint formulation of Algorithm 3 has some computational overhead compared to Algorithm 2. The setup stage requires an additional Cholesky factorization of

the $N \times N$ matrix M, which costs $\mathcal{O}(N^{3/2})$. For the QR factorization, Y is now $N \times l$, so this costs $\mathcal{O}(l^2 N)$, compared to $\mathcal{O}(l^2\sqrt{N})$ for rSVD on K. Computing $B = Q^T K^*$ requires solving $M\tilde{q}_i = q_i$ l times, costing $\mathcal{O}(lN \log N)$ (compared to $\mathcal{O}(l\sqrt{N} \log \sqrt{N})$, which was negligible). The SVD of $B \in \mathbb{R}^{l \times N_b}$ costs $\mathcal{O}(l^2\sqrt{N})$, which is cheaper than $\mathcal{O}(l^2 N)$ for rSVD on K. Overall, the asymptotic cost of Algorithm 3 is the same as Algorithm 2, which is given in Equation (30).

In Table 5 (t_1/t), we see approximately 30% computational overhead for $k = 10$, 20% for $k = 50$, and 10% for $k = 200$. This matches the theoretical overhead, which reduces for larger k when the cost $\mathcal{O}(l^2 N)$ of the SVD in Algorithm 2 takes up a larger share.

To recover the SVD of K from the SVD of K^* (Section 3.2.1), forming $A = M_\partial^{-1}\bar{V}$ involves k triangular solves with M_∂ ($\mathcal{O}(k\sqrt{N} \log \sqrt{N})$), and forming $B = M\bar{U}$ is a sparse matrix-matrix product ($\mathcal{O}(kN)$): both are cheap and can be disregarded. Their respective QR factorizations add up to $\mathcal{O}(k^2\sqrt{N}) + \mathcal{O}(k^2 N) = \mathcal{O}(k^2 N)$. The SVD of $R_A \bar{\Sigma} R_B^T \in \mathbb{R}^{k \times k}$ is $\mathcal{O}(k^3)$, which is negligible compared to $\mathcal{O}(k^2 N)$, since $k \ll N$.

This is in correspondence with the reported computational overhead t_2/t in Table 5: as k increases, the extra cost $\mathcal{O}(k^2 N)$ of the recovery procedure and the reduced cost $\mathcal{O}(l^2\sqrt{N})$ of the SVD of B balance out, leading to approximately 30% extra computational time using rSVD on K^* and recovery of the SVD, compared to the direct approach of rSVD on K.

6.1.3 Analysis of reconstruction error

At target rank $k = 50$, rSVD reconstruction errors were within 0.5% to 4% of the exact Tikhonov solution in Euclidean norm and AUC-IoU (4% to 39% in EMD), with visually indistinguishable reconstructions (Section 5.3.3, Figure 11).

To understand these results, we inspect the structure of $Y = K\Psi$. Following the setup of Halko et al. [23, Sec. 9], let $K = U\Sigma V^T$ be the SVD of K, and partition Σ as $\Sigma = \mathrm{diag}(\Sigma_1, \Sigma_2)$, where Σ_1 contains the k first singular values of K, and Σ_2 the remainder. Correspondingly, partition the right singular vectors as $V = [V_1\ V_2]$, and write K as

$$K = U \begin{bmatrix} \Sigma_1 & \\ & \Sigma_2 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix}. \tag{31}$$

Right-multiplying by Ψ and setting $\Psi_1 = V_1^T\Psi$ and $\Psi_2 = V_2^T\Psi$ gives

$$Y = U \begin{bmatrix} \Sigma_1\Psi_1 \\ \Sigma_2\Psi_2 \end{bmatrix}.$$

Here, $\Sigma_1\Psi_1$ is the contribution from the k dominant singular components we wish to recover, while $\Sigma_2\Psi_2$ comes from the truncated tail and acts as a perturbation.

By Theorem 10.5 in [23], if $\Psi \in \mathbb{R}^{N \times l}$ is a random Gaussian matrix, then

$$\mathbb{E}\|K - K_k\|_F \leq \gamma_k \sqrt{\sum_{i=k+1}^{N_b} \sigma_i^2}, \tag{32}$$

where $\mathbb{E}$ denotes the expected value, $\gamma_k = \sqrt{1 + k/(p-1)}$, and K_k is the rank-k approximation obtained from the rSVD (Algorithm 2 or 3).

When the singular values of K decay rapidly, the perturbation $\Sigma_2 \Psi_2$ remains low and the sum in Equation (32) is small, resulting in a good rank-k approximation. If the singular values decay more slowly, $\sum_{i=k+1}^{N_b} \sigma_i^2$ will be larger, and the expected error increases.

Since K has rapidly decaying singular values (Figure 4), a target rank $k \ll \text{rank}(K) = N_b$ yields an accurate rank-k approximation: the main actions of K are captured in $\Sigma_1 \Psi_1$. This argument transfers to the weighted Tikhonov problem in Equation (22). Both K and W are approximated accurately at $k = 50$, keeping the reconstruction error within 4% (Euclidean, AUC-IoU). Since the random sampling introduces some noise, the EMD shows a higher error, as it is more sensitive to low-amplitude noise.

6.1.4 Reconstruction error using the adjoint operator

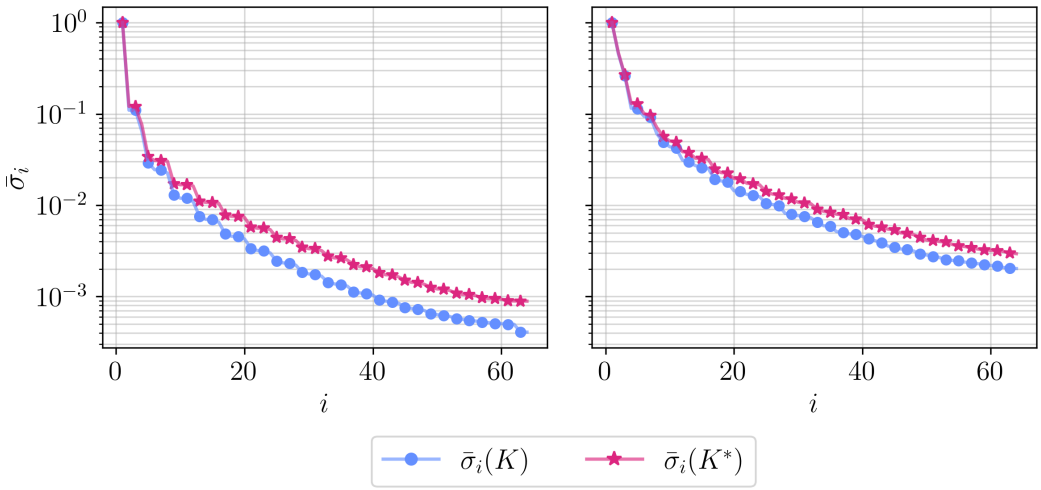


Figure 29: Comparison of the normalized singular values $\bar{\sigma}_i(K)$ and $\bar{\sigma}_i(K^*)$ of the forward and adjoint operators, respectively. Left: Problem I and II. Right: Problem III.

Using rSVD on K gives slightly lower reconstruction error than rSVD on K^* (Section 5.3.8). The quality of the random sampling $Y = K\Psi$ versus $Y' = K^*\Psi'$ depends on the spectral decay of the operators. Figure 29 shows that the singular values of K^* have a slower decay than those of K, and we therefore expect a higher approximation error by Equation (32). The matrices K and K^T have the same

singular values, so the difference originates in the mass matrix factors of the relation $K^* = M^{-1}K^T M_\partial$ (Equation (16)).

6.1.5 Effect of the test vector distribution

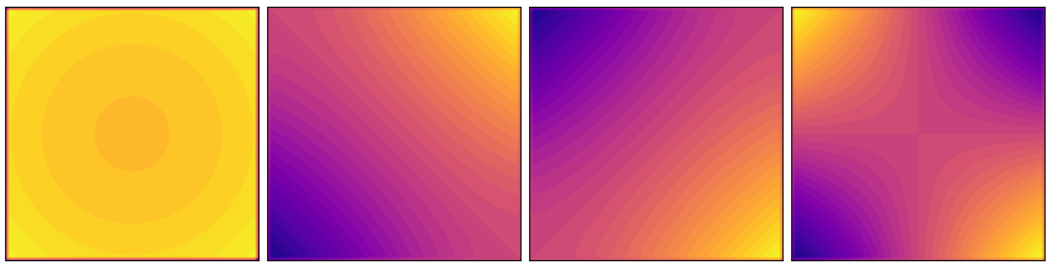


Figure 30: The first four right singular vectors $v_1, \ldots, v_4$ (shown in the same order) of the forward operator K from Problem I.

Some test vector distributions slightly reduce reconstruction error, with the Peak test vectors giving lower EMD than Gaussian or Rademacher (Section 5.3.4).

Consider a single sample $y = K\psi = U\Sigma V^T\psi$. In the best case, $\psi = av_1$ with $a \neq 0$: then $V^T(av_1) = ae_1$, $K(av_1) = a\sigma_1 u_1$, and the QR factorization gives $Q = u_1$. More generally, choosing k linearly independent $\psi_i \in \text{span}\{v_1, \ldots, v_k\}$ gives $\text{range}(Q) = \text{span}\{u_1, \ldots, u_k\}$ and the rank-k projection QQ^TK recovers the exact rank-k SVD of K.

Figure 30 plots the first four right singular vectors of K. We see that they are smooth, more closely matching the Peak test vectors than the Gaussian or Rademacher test vectors, potentially explaining the small improvement in error.

6.1.6 Independence from mesh resolution

The reconstruction error is independent of the mesh resolution N for fixed k (Section 5.3.5). The PDE solution at the boundary is smooth, so finer discretization produces more samples of the same smooth function without introducing new information about the source.

The continuous forward operator $\mathcal{K}$ has rapidly decaying singular values (Section 2.1.1), and the discrete K inherits this decay. As N increases and $\text{rank}(K) = N_b$ grows, the additional singular values are small with little new information.

The target rank k can therefore be selected on a coarse mesh, where rSVD is inexpensive to run, and reused unchanged as the mesh is refined. This is a practical method for tuning k when the final computation on a fine mesh is expensive.

6.1.7 Estimation of singular values

Let $\hat{P} = QQ^T$ with Q produced by Algorithm 2. This is an orthogonal projection onto $\text{range}(Q)$, and $\hat{P}K$ is the projected matrix from which the rSVD approximation

is obtained. The projector satisfies $\|\hat{P}z\| \leq \|z\|$, meaning it shrinks vectors, so $\|\hat{P}Kx\| \leq \|Kx\|$ for every x. Consequently, $\hat{P}K$ stretches every vector at most as much as K does, and singular values measure exactly this stretching. This is what we saw in Section 5.3.6. By choosing a higher oversampling parameter p, range(Q) more closely matches range(K) (in particular, the k first left singular vectors), and the underestimation reduces. Algorithm 3 gave a worse approximation (Section 5.3.8), so the underestimation increases.

The captured subspace range(Q) depends on the random test vectors ψ_i, and so does the degree of underestimation. This is an active area of research. Lemma 4.2 and Theorem 4.3 of Gu [24] provide lower bounds for the power-iteration method, which reduces to the plain rSVD used in Algorithm 2 when $q = 0$.

6.1.8 Reconstructions under noisy data

Choosing k too large worsens reconstruction quality on noisy data (Section 5.3.9), and the target rank acts as a regularization parameter in the same manner as the truncation rank of tSVD.

For a noisy measurement $\tilde{y} = y + \epsilon$, $y = Kx$, we can write the minimum-norm least-squares solution as $x^* = K^+\tilde{y} = K^+y + K^+\epsilon$ (Section 2.4.4). Expanding in terms of the SVD components,

$$x^* = \sum_{i=1}^{N_b} \left(\frac{u_i^T y + u_i^T \epsilon}{\sigma_i} \right) v_i,$$

we see that the i-th component depends on the exact term $u_i^T y$ and the noise term $u_i^T \epsilon$. Components where $|u_i^T y| \leq |u_i^T \epsilon|$ are dominated by noise, and the $\frac{1}{\sigma_i}$ factor amplifies this contribution. Assuming that the Picard condition [8] holds, $u_i^T y$ decays to zero as i increases while $u_i^T \epsilon$ stays approximately constant, so for large i every component is noise-dominated and amplified by a growing $\frac{1}{\sigma_i}$.

By truncating the $N_b - k$ terms of the solution, we remove the noisy SVD components. We wish to choose k such that $|u_k^T y| > |u_k^T \epsilon|$ while $|u_{k+1}^T y| < |u_{k+1}^T \epsilon|$, removing only the noise-dominated SVD components $k+1, k+2, \ldots, N_b$.

For the weighted Tikhonov solution, the reconstruction already has some regularization through λ, but the argument transfers. We found that the optimal k values are approximately equal for both the rSVD and the tSVD. The rank is problem-dependent, and for our problems ranged from $k = 45$ to $k = 85$ by Euclidean norm. The EMD preferred a smaller k (20 to 30), since it is more sensitive to unwanted noise in the reconstruction, while the AUC-IoU generally preferred larger k (≥ 60), since it focuses on segmentation overlap, which is less affected by noise.

6.2 Dynamical low-rank approximation

6.2.1 Convergence of DLRA-CG

Replacing the steepest descent search directions with directions $\{D^{(i)}\}$ from CG increased convergence significantly: DLRA-CG reached the relative error $e^{(i)} < 0.85$ approximately 210 times faster than steepest descent (Section 5.4.1).

In an unconstrained setting, the convergence of CG and steepest descent depends on the condition number $\kappa = \kappa(H)$, with larger κ leading to slower convergence [43]. If x^* is the unconstrained minimizer of $\Phi(x)$ and $x^{(i)}$ the i-th iterate of CG, then by Theorem 6.29 in Saad [43],

$$\|x^* - x^{(i)}\|_H \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^i \|x^* - x^{(0)}\|_H. \tag{33}$$

If κ is large, then $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \approx 1 - \frac{2}{\sqrt{\kappa}}$, compared to the slower convergence of steepest descent which converges as approximately $1 - \frac{2}{\kappa}$ [43].

The Hessian $H = F + S$ has a low-rank $F = K^T M_\partial K$ plus a sparse $S = \lambda^2 W^T M W$ structure (Section 4.5.1). The rapid decay of singular values of K makes $\kappa(H)$ depend on λ: the small eigenvalues of H are mostly determined by S, whose eigenvalues are small when λ is small. For example, at $\lambda = 0.1$, $\kappa(H) \approx 10^3$, and at $\lambda = 10^{-4}$, $\kappa(H) \approx 10^9$.

The same argument partially transfers to DLRA-CG, as demonstrated by Table 8, since $\{D^{(i)}\}$ is generated by the same CG recurrence as in the unconstrained case. A formal convergence analysis of DLRA-CG that accounts for the projection and truncation remains an open problem.

6.2.2 Semi-convergence and restarting

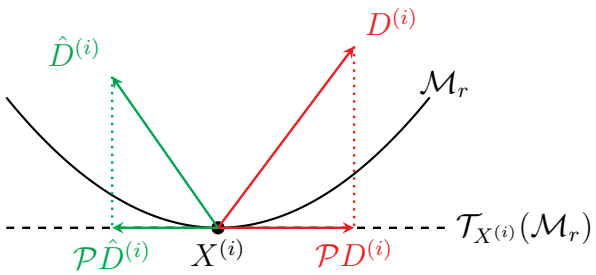


Figure 31: The CG search direction $D^{(i)}$ may disagree with the steepest-descent direction $\hat{D}^{(i)} = -\nabla\Phi(X^{(i)})$, so that their projections $\mathcal{P}D^{(i)}$ and $\mathcal{P}\hat{D}^{(i)}$ onto the tangent space $\mathcal{T}_{X^{(i)}}(\mathcal{M}_r)$ point in very different directions.

DLRA-CG shows semi-convergence: initially the relative error reduces, and then it rises again before stalling (Section 5.4.1). Restarted DLRA-CG avoids this, but shows slower convergence.

As discussed in Section 4.4.1, the DLRA-CG algorithm constructs $(\alpha_i, G^{(i)}, D^{(i)})$ independently of $X^{(i)}$. The search directions $\{D^{(i)}\}$ follow the trajectory of the full-rank solution $Y^{(i)}$, meaning $G^{(i)} = \nabla\Phi(Y^{(i)})$. When i increases, $X^{(i)}$ and $Y^{(i)}$ may drift apart so that $G^{(i)}$ and the true gradient $\hat{G}^{(i)} = \nabla\Phi(X^{(i)})$ increasingly disagree. The projections of $D^{(i)}$ and $\hat{D}^{(i)} = -\hat{G}^{(i)}$,

$$\mathcal{P}D^{(i)} = \mathcal{P}_{X^{(i)}}(D^{(i)}), \quad \mathcal{P}\hat{D}^{(i)} = \mathcal{P}_{X^{(i)}}(\hat{D}^{(i)}),$$

may disagree to the point that the relative error $e^{(i)}$ rises again, resulting in semi-convergence. This is illustrated in Figure 31.

Restarting sets $D^{(i)} = \hat{D}^{(i)}$, so the projected step $\mathcal{P}D^{(i)} = \mathcal{P}\hat{D}^{(i)}$ is the descent direction at $X^{(i)}$. As the iterations continue and $X^{(i)}$ drifts away from $Y^{(i)}$, this alignment gradually breaks down until the next restart. When the restart period N_r is too small, the convergence of DLRA-CG will approach the steepest descent rate (at $N_r = 1$ they are equal), but choosing N_r too large gives $X^{(i)}$ and $Y^{(i)}$ time to drift apart, so that semi-convergence re-appears.

An adaptive selection of N_r could be considered, for example only restarting once the angle or relative difference between $G^{(i)}$ and the true gradient $\hat{G}^{(i)}$ is above some threshold.

6.2.3 Rank as a regularization parameter

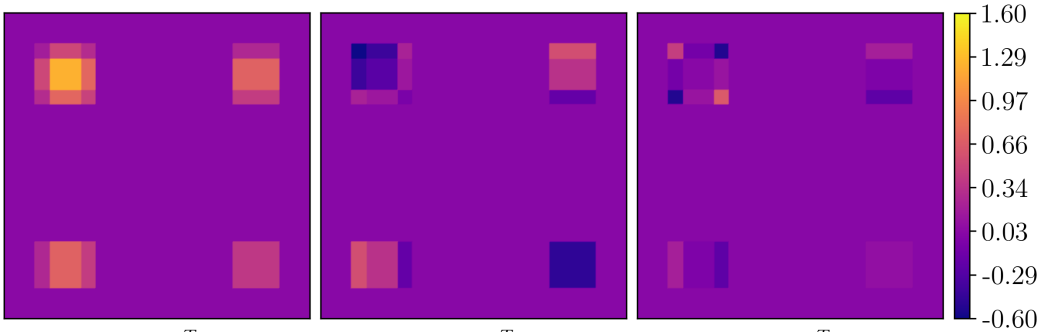


Figure 32: The three individual rank-1 matrices whose sum makes up the source of Problem II. Each rank-1 matrix is the outer product of a left and a right singular vector, scaled by their corresponding singular value.

The low-rank solutions outperformed exact weighted Tikhonov reconstructions (Section 5.4.2), but were sensitive to the choice of r: choosing r slightly above optimal gave reconstructions resembling the full-rank case, while choosing r too small introduced spurious sources for Problem II (Section 5.4.3).

The sources in Problems I and II are of low rank and already live on $\mathcal{M}_r$. Each square source factors as an outer product of two vectors, giving a rank-1 matrix. The single source of Problem I is therefore of rank 1, and the three sources of

Problem II sum to a matrix of rank 3, see Figure 32. However, we found that a rank-2 approximation of this source yielded better reconstruction error. We interpret this as the rank constraint counteracting Tikhonov's oversmoothing: the discarded component is the most heavily smoothed but the least important of the three, so removing it is beneficial.

In practice, the residual as a function of rank served as a practical method for choosing r in the case of multiple sources (Problem II), but there was no obvious connection between the residual and the optimal rank for the simple rank-1 source of Problem I (Section 5.4.4).

6.2.4 Effect of preconditioning

We saw that preconditioning sped up convergence, but also raised the reconstruction error (Section 5.4.5), and the iterates diverged when using an IC + Woodbury preconditioner together with restarting.

The convergence of CG depends on the condition number $\kappa(H)$. By using a preconditioner $P \approx H$, the condition number $\kappa(P^{-1}H)$ is reduced, and the convergence is sped up by Equation (33). As discussed in Section 6.2.1, H has a large condition number, which makes preconditioning effective.

In the setting of DLRA-PCG, a stronger P accelerates the convergence of the iterates $Y^{(i)}$ so that they reach the full-rank weighted Tikhonov solution in fewer iterations, leaving $X^{(i)}$ further behind on $\mathcal{M}_r$. This raises the error of the final $X^{(i)}$. In the case of the perfect preconditioner $P^{-1} = H^{-1}$, $Y^{(i)}$ reaches the full-rank solution in a single step, and $X^{(i)}$ is moved a single step in the projected direction and truncated to $\mathcal{M}_r$.

When using the IC preconditioner, the higher final error was resolved by restarting. With the IC + Woodbury preconditioner and restarting, however, the iterates $X^{(i)}$ diverge. Since $\mathcal{M}_r$ is non-convex [34], $X^{(i)}$ may converge to a local minimum, as demonstrated in Section 5.4.6, but we have no complete explanation.

7 Conclusion

We studied a source identification problem with an elliptic PDE-constrained forward operator which suffers from rapid decay of singular values and a large nullspace, leading to ill-posedness. Standard Tikhonov regularization gives biased solutions concentrated near the domain boundary, and smooths solutions such that distinct sources blur together.

We introduced a matrix-free rSVD approach that computes a rank-k approximation to the SVD of the forward operator, avoiding explicit formation, and making Elvetun and Nielsen's weighted Tikhonov approach applicable at large scale. We also introduced the DLRA-CG method to accelerate the convergence of DLRA and efficiently recover sparse sources.

The matrix-free rSVD solved previously computationally infeasible problems without significant loss in reconstruction quality. For the problems studied, a target rank of approximately 50 sufficed, independent of the mesh size N. On noisy data, the target rank acted as additional regularization, yielding comparable results to truncated SVD. We also applied DLRA-CG to recover low-rank solutions, converging up to approximately 210 times faster than DLRA with steepest descent in the tested problems. The solutions achieved lower reconstruction error and separated distinct sources more clearly than full-rank weighted Tikhonov. DLRA-CG showed semi-convergence, which was solved using restarting, at the cost of slower convergence. Preconditioning (DLRA-PCG) achieved significantly faster convergence but increased the reconstruction error. Restarting partially resolved the higher error but produced unstable results for the IC + Woodbury preconditioner.

The matrix-free rSVD was tested on a single elliptic PDE with simple two-dimensional geometries and square sources, and its performance on other operators or domains remains untested. For operators with a slow decay of singular values, the target rank k must be larger, reducing the feasibility of the method. The random nature of the method gives varying reconstructions between runs, but this can be controlled by the oversampling parameter p. Applying the method to the adjoint operator K^* gave mixed results: at $k = 25$, EMD error increased by 9-31% due to higher background noise, while Euclidean error stayed within $\pm 1\%$ across all k and AUC-IoU within $\pm 2\%$ for $k \geq 25$.

The DLRA-CG method is sensitive to the choice of rank r, where too small a rank gives incorrect solutions while a rank only slightly above optimal gives a reconstruction resembling the full-rank case. Without restarting, it is unclear when to stop to avoid semi-convergence. However, using DLRA-CG with restarting requires choosing a restart period: too short and convergence approaches steepest descent, too long and semi-convergence returns. Preconditioning introduced further complications, where iterates converged toward non-optimal solutions, with the effect more severe for preconditioners that closely approximated the Hessian.

Extending the matrix-free rSVD to other PDE types, three-dimensional domains,

more complex sources, and other geometries would establish how broadly the results generalize. The method could be extended using power iteration, like Algorithm 4.3 in [23], for problems of slow spectral decay, or using an adaptive rank selection based on a specified tolerance, like Algorithm 4.2 in [23]. It is however unclear how a tolerance for K transfers to the reconstruction error. Further investigation is needed to determine under which conditions the adjoint approach is preferable.

For DLRA-CG, the rank-adaptive variant of Schotthöfer et al. [11] could be explored, which removes the need for manual rank selection by automatically adapting the rank of the iterates based on a threshold. Stopping criteria such as the discrepancy principle or the L-curve [8] could be used to address semi-convergence. Establishing a convergence theory for DLRA-CG remains an open problem, particularly regarding the effect of preconditioning.

Together, these contributions provide efficient low-rank methods for sparse source identification in large-scale PDE-constrained inverse problems.

References

[1] W. A. Strauss. Partial Differential Equations: An Introduction. John Wiley & Sons, 2007. ISBN: 978-0-470-05456-7.

[2] L. C. Evans. Partial Differential Equations. 2nd Ed. American Mathematical Society, 2010. ISBN: 978-1-4704-6942-9.

[3] G. A. Pavliotis. Stochastic Processes and Applications: Diffusion Processes, the Fokker-Planck and Langevin Equations. Vol. 60. Texts in Applied Mathematics. New York, NY: Springer, 2014. ISBN: 978-1-4939-1322-0 978-1-4939-1323-7. DOI: 10.1007/978-1-4939-1323-7.

[4] B. F. Nielsen, M. Lysaker, and P. Grøttum. Computing Ischemic Regions in the Heart with the Bidomain Model–First Steps towards Validation. *IEEE transactions on medical imaging* **32**(6), 1085–1096, 2013. DOI: 10.1109/TMI.2013.2254123.

[5] R. Elul. The Genesis of the EEG. *International Review of Neurobiology* **15**, 227–272, 1971. DOI: 10.1016/s0074-7742(08)60333-5.

[6] C. J.S. Alves, B. A. Jalel, and J. Mohamed. Recovery of Cracks Using a Point-Source Reciprocity Gap Function. *Inverse Problems in Science and Engineering* **12**(5), 519–534, 2004. DOI: 10.1080/1068276042000219912.

[7] S. Baillet, J. Mosher, and R. Leahy. Electromagnetic Brain Mapping. *Signal Processing Magazine, IEEE* **18**, 14–30, 2001. DOI: 10.1109/79.962275.

[8] P. Hansen. Discrete Inverse Problems: Insight and Algorithms. Fundamentals of Algorithms. Society for Industrial and Applied Mathematics, 2010. ISBN: 978-0-89871-696-2.

[9] O. L. Elvetun and B. F. Nielsen. A Regularization Operator for Source Identification for Elliptic PDEs. DOI: 10.48550/arXiv.2005.09444. arXiv: 2005.09444 [math]. 2020.

[10] O. L. Elvetun and B. F. Nielsen. Modified Tikhonov Regularization for Identifying Several Sources. DOI: 10.48550/arXiv.2011.04394. arXiv: 2011.04394 [math]. 2020.

[11] S. Schotthöfer, E. Zangrando, J. Kusch, G. Ceruti, and F. Tudisco. Low-Rank Lottery Tickets: Finding Efficient Low-Rank Neural Networks via Matrix Differential Equations. DOI: 10.48550/arXiv.2205.13571. arXiv: 2205.13571 [cs]. 2022.

[12] D. P. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. DOI: 10.48550/arXiv.1412.6980. arXiv: 1412.6980 [cs]. 2017.

[13] E. Chong, W. Lu, and S. Zak. An Introduction to Optimization: With Applications to Machine Learning. 5th ed. Wiley, 2023. ISBN: 978-1-119-87763-9.

[14] J. Nocedal and S. Wright. Numerical Optimization. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006. ISBN: 978-0-387-40065-5.

[15] M. Hestenes and E. Stiefel. Methods of Conjugate Gradients for Solving Linear Systems. *Journal of Research of the National Bureau of Standards* **49**(6), 409, 1952. DOI: 10.6028/jres.049.044.

[16] R. Fletcher and C. M. Reeves. Function Minimization by Conjugate Gradients. *The Computer Journal* **7**(2), 149–154, 1964. DOI: 10.1093/comjnl/7.2.149.

[17] H. Engl, M. Hanke, and A. Neubauer. Regularization of Inverse Problems. Mathematics and Its Applications. Springer Netherlands, 1996. ISBN: 978-0-7923-4157-4.

[18] M. G. Larson and F. Bengzon. The Finite Element Method: Theory, Implementation, and Applications. Springer Science & Business Media, 2013. ISBN: 978-3-642-33287-6.

[19] E. Kreyszig. Introductory Functional Analysis with Applications. Wiley, 1989. ISBN: 978-0-471-50459-7.

[20] D. C. Lay, S. R. Lay, and J. J. McDonald. Linear Algebra and Its Applications. Sixth edition, global edition. Boston: Pearson, 2022. ISBN: 978-1-292-35121-6.

[21] C. Eckart and G. Young. The Approximation of One Matrix by Another of Lower Rank. *Psychometrika* **1**(3), 211–218, 1936. DOI: 10.1007/BF02288367.

[22] G. H. Golub and C. F. Van Loan. Matrix Computations - 4th Edition. Philadelphia, PA: Johns Hopkins University Press, 2013. DOI: 10.1137/1.9781421407944.

[23] N. Halko, P. G. Martinsson, and J. A. Tropp. Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions. *SIAM Review* **53**(2), 217–288, 2011. DOI: 10.1137/090771806.

[24] M. Gu. Subspace Iteration Randomization and Singular Value Problems. *SIAM Journal on Scientific Computing* **37**(3), A1139–A1173, 2015. DOI: 10.1137/130938700.

[25] Y. Diouane, S. Gürol, A. S. D. Perrotolo, and X. Vasseur. A General Error Analysis for Randomized Low-Rank Approximation Methods. DOI: 10.48550/arXiv.2206.08793. arXiv: 2206.08793 [math]. 2022.

[26] V. Rokhlin and M. Tygert. A Fast Randomized Algorithm for Overdetermined Linear Least-Squares Regression. *Proceedings of the National Academy of Sciences* **105**(36), 13212–13217, 2008. DOI: 10.1073/pnas.0804869105.

[27] O. L. Elvetun and B. F. Nielsen. Weighted Sparsity Regularization for Source Identification for Elliptic PDEs. DOI: 10.48550/arXiv.2012.11280. arXiv: 2012.11280 [math]. 2023.

[28] J. Dongarra, M. Gates, A. Haidar, J. Kurzak, P. Luszczek, S. Tomov, and I. Yamazaki. The Singular Value Decomposition: Anatomy of Optimizing an Algorithm for Extreme Scale. *SIAM Review* **60**(4), 808–865, 2018. DOI: 10.1137/17M1117732.

[29] J. A. Tropp. ACM 204: Randomized Algorithms for Matrix Computations. Tech. rep. Pasadena, CA: California Institute of Technology, 2020. DOI: 10.7907/nwsv-df59.

[30] A. George. Nested Dissection of a Regular Finite Element Mesh. *SIAM Journal on Numerical Analysis* **10**(2), 345–363, 1973. DOI: 10.1137/0710032.

[31] A. George and J. W. H. Liu. Computer Solution of Large Sparse Positive Definite Systems. Vol. 26, 1984. DOI: 10.1137/1026055.

[32] R. J. Lipton, D. J. Rose, and R. E. Tarjan. Generalized Nested Dissection. *SIAM Journal on Numerical Analysis* **16**(2), 346–358, 1979. DOI: 10.1137/0716027.

[33] M. do Carmo. Riemannian Geometry. Mathematics (Boston, Mass.) Birkhäuser, 1992. ISBN: 978-3-7643-3490-1.

[34] P.-A. Absil, R. Mahony, and R. Sepulchre. Optimization Algorithms on Matrix Manifolds. Vol. 78, 2008. ISBN: 978-0-691-13298-3. DOI: 10.1515/9781400830244.

[35] O. Koch and C. Lubich. Dynamical Low-rank Approximation. *SIAM Journal on Matrix Analysis and Applications* **29**(2), 434–454, 2007. DOI: 10.1137/050639703.

[36] C. Lubich and I. Oseledets. A Projector-Splitting Integrator for Dynamical Low-Rank Approximation. DOI: 10.48550/arXiv.1301.1058. arXiv: 1301.1058 [math]. 2013.

[37] E. Kieri, C. Lubich, and H. Walach. Discretized Dynamical Low-Rank Approximation in the Presence of Small Singular Values. *SIAM Journal on Numerical Analysis* **54**(2), 1020–1038, 2016. DOI: 10.1137/15M1026791.

[38] J. Kusch, L. Einkemmer, and G. Ceruti. On the Stability of Robust Dynamical Low-Rank Approximations for Hyperbolic Problems. *SIAM Journal on Scientific Computing* **45**(1), A1–A24, 2023. DOI: 10.1137/21M1446289.

[39] G. Ceruti, J. Kusch, and C. Lubich. A Parallel Rank-Adaptive Integrator for Dynamical Low-Rank Approximation. *SIAM Journal on Scientific Computing* **46**(3), B205–B228, 2024. DOI: 10.1137/23M1565103.

[40] G. Ceruti, J. Kusch, and C. Lubich. A Rank-Adaptive Robust Integrator for Dynamical Low-Rank Approximation. DOI: 10.48550/arXiv.2104.05247. arXiv: 2104.05247 [math]. 2021.

[41] G. Ceruti and C. Lubich. An Unconventional Robust Integrator for Dynamical Low-Rank Approximation. DOI: 10.48550/arXiv.2010.02022. arXiv: 2010.02022 [math]. 2020.

[42] S. Schotthöfer, T. Klein, and J. Kusch. A Geometric Framework for Momentum-Based Optimizers for Low-Rank Training. *Advances in Neural Information Processing Systems* ed. by D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen. **38**. Curran Associates, Inc., 90050–90080, 2025.

[43] Y. Saad. Iterative Methods for Sparse Linear Systems. Society for Industrial and Applied Mathematics, 2003. ISBN: 978-0-89871-534-7.

[44] M. J. D. Powell. Restart Procedures for the Conjugate Gradient Method. *Mathematical Programming* **12**(1), 241–254, 1977. DOI: 10.1007/BF01593790.

[45] P. Concus, G. H. Golub, and D. P. O'Leary. A Generalized Conjugate Gradient Method for the Numerical Solution of Elliptic Partial Differential Equations. *Sparse Matrix Computations*. Ed. by J. R. BUNCH and D. J. ROSE. Academic Press, 309–332, 1976. ISBN: 978-0-12-141050-6. DOI: 10.1016/B978-0-12-141050-6.50023-4.

[46] M. Woodbury. Inverting Modified Matrices. Memorandum Report / Statistical Research Group, Princeton. Department of Statistics, Princeton University, 1950.

[47] C. R. Harris et al. Array Programming with NumPy. *Nature* **585**(7825), 357–362, 2020. DOI: 10.1038/s41586-020-2649-2.

[48] P. Virtanen et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* **17**(3), 261–272, 2020. DOI: 10.1038/s41592-019-0686-2.

[49] M. Alnæs, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. Rognes, and G. Wells. The FEniCS Project Version 1.5. **3**, 2015. DOI: 10.11588/ans.2015.100.20553.

[50] A. Logg, K.-A. Mardal, and G. Wells, eds. Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book. Vol. 84. Lecture Notes in Computational Science and Engineering. Berlin, Heidelberg: Springer, 2012. ISBN: 978-3-642-23098-1 978-3-642-23099-8. DOI: 10.1007/978-3-642-23099-8.

[51] S. J. Montgomery-Smith. The Distribution of Rademacher Sums. *Proceedings of the American Mathematical Society* **109**, 517–522, 1990. DOI: 10.1090/S0002-9939-1990-1013975-0.

[52] A. Quarteroni. Numerical Models for Differential Problems. Milano: Springer Milan, 2014. ISBN: 978-88-470-5521-6 978-88-470-5522-3. DOI: 10.1007/978-88-470-5522-3.

[53] C.-J. Lin and J. J. Moré. Incomplete Cholesky Factorizations with Limited Memory. *SIAM Journal on Scientific Computing* **21**(1), 24–45, 1999. DOI: 10.1137/S1064827597327334.

[54] T. A. Davis. Direct Methods for Sparse Linear Systems. Fundamentals of Algorithms. Society for Industrial and Applied Mathematics, 2006. ISBN: 978-0-89871-613-9. DOI: 10.1137/1.9780898718881.

[55] E. Polak and G. Ribiere. Note sur la convergence de méthodes de directions conjuguées. *Revue française d'informatique et de recherche opérationnelle. Série rouge* **3**(R1), 35–43, 1969.

[56] Y. Rubner, C. Tomasi, and L. Guibas. A Metric for Distributions with Applications to Image Databases. *Sixth International Conference on Computer Vision* 59–66, 1998. DOI: 10.1109/ICCV.1998.710701.

[57] S. Raschka and V. Mirjalili. Python Machine Learning: Machine Learning and Deep Learning with Python, Scikit-Learn, and TensorFlow 2. 2nd ed. Birmingham Mumbai: Packt Publishing, 2019. ISBN: 978-1-78712-593-3.

A Appendix

A.1 Functional analysis and the FEM

Lebesgue space. Let $\Omega \subset \mathbb{R}^d$ be an open set and $\mathbb{F}$ denote either $\mathbb{R}$ or $\mathbb{C}$. The space $L^2(\Omega)$, often referred to as the Lebesgue space of square integrable functions, is defined as

$$L^2(\Omega) = \left\{ f : \Omega \to \mathbb{F} \;\middle|\; \int_\Omega |f(x)|^2 \, dx < \infty \right\},$$

with inner product

$$\langle f, g \rangle_{L^2(\Omega)} = \int_\Omega f(x)\overline{g(x)} \, dx.$$

The space $L^2(\Omega)$ is a Hilbert space, i.e., a complete inner product space. For details, see Section 3 in Kreyszig [19].

Sobolev space. The space $H^1(\Omega)$, known as a first-order Sobolev space, is defined as

$$H^1(\Omega) = \left\{ f \in L^2(\Omega) \;\middle|\; \frac{\partial f}{\partial x_i} \in L^2(\Omega),\ i = 1, \ldots, d \right\},$$

with inner product

$$\langle f, g \rangle_{H^1(\Omega)} = \langle f, g \rangle_{L^2(\Omega)} + \langle \nabla f, \nabla g \rangle_{L^2(\Omega)}.$$

The space $H^1(\Omega)$ is a Hilbert space and a subspace of $L^2(\Omega)$. See Section 2 in Quarteroni for further details [52].

The stiffness and the mass matrix. Let $V_h \subset H^1(\Omega)$ be the finite element space defined in Equation (7). Under the basis $\{\phi_i\}_{i=1}^N$, the stiffness matrix $A_h \in \mathbb{R}^{N \times N}$ and mass matrix $M \in \mathbb{R}^{N \times N}$ are defined as

$$(A_h)_{ij} = \int_\Omega (\sigma \nabla \phi_j \cdot \nabla \phi_i + c\phi_j \phi_i) \, dx, \quad M_{ij} = \int_\Omega \phi_j \phi_i \, dx.$$

Both matrices are symmetric positive definite (Appendix A.3) under the standing assumptions that σ is positive definite and $c > 0$, see Theorem 3.4 and Theorem 4.10 in Larson and Bengzon [18]. The boundary mass matrix $M_\partial \in \mathbb{R}^{N_b \times N_b}$ is given by

$$(M_\partial)_{ij} = \int_{\partial\Omega} \phi_i \phi_j \, ds,$$

where the indices i, j tabulate over the boundary indices of the mesh. Like the mass matrix M, the boundary mass matrix M_∂ is SPD [18].

A_h, M and M_∂ are sparse matrices, since $(A_h)_{ij}$, M_{ij} and $(M_\partial)_{ij}$ are non-zero only if ϕ_i and ϕ_j share an element in the mesh.

The assembly of stiffness matrices and mass matrices in two dimensions can be done through methods of numerical integration, as described in Section 3.5 in Larson and Bengzon [18].

A.2 Derivation of the adjoint formulation

Consider the forward operator $\mathcal{K} = \mathcal{T} \circ \mathcal{S}$ given by Equation (5), and the variational formulation of $\mathcal{S}$ by Equation (3). Define the bilinear form

$$a(u,\, v) = \int_\Omega (\sigma \nabla u \cdot \nabla v + cuv)\, dx,$$

then, the variational formulation of $\mathcal{S}$ reads as follows: find $u \in H^1(\Omega)$ such that

$$a(u,\, v) = \int_\Omega fv\, dx. \tag{A1}$$

For some $g \in L^2(\partial\Omega)$, we seek $w = \mathcal{K}^* g$. By the definition of $\mathcal{K}^*$,

$$\langle \mathcal{K} f,\, g \rangle_{L^2(\partial\Omega)} = \langle f,\, \mathcal{K}^* g \rangle_{L^2(\Omega)}.$$

and since $\mathcal{K} f = u|_{\partial\Omega}$, we have that

$$\langle \mathcal{K} f,\, g \rangle_{L^2(\partial\Omega)} = \int_{\partial\Omega} ug\, ds. \tag{A2}$$

In Equation (A1), choose $v = w = \mathcal{K}^* g$. We see that

$$a(u, w) = \int_\Omega fw\, dx = \langle f,\, \mathcal{K}^* g \rangle_{L^2(\Omega)}. \tag{A3}$$

Inserting Equations (A2) and (A3) into the definition of $\mathcal{K}^*$, and using the identity $a^*(v,\, u) = a(u,\, v)$ (by the definition of a^* [19]),

$$a^*(w,\, u) = \int_{\partial\Omega} gu\, ds. \tag{A4}$$

Since Equation (A4) must hold for all $u \in H^1(\Omega)$, it defines a variational problem for w. And since a is symmetric (σ is symmetric), $a^*(w,\, u) = a(u,\, w) = a(w,\, u)$, we get that Equation (A4) reduces to: find $w \in H^1(\Omega)$ such that

$$a(w,\, u) = \int_{\partial\Omega} gu\, ds \quad \text{for all } u \in H^1(\Omega),$$

where

$$a(w, u) = \int_\Omega (\sigma \nabla w \cdot \nabla u + cwu)\, dx.$$

This is the variational formulation of Equation (15) with $u := v$.

A.3 Linear algebra and matrix decompositions

The definitions and properties throughout this subsection follow Golub and Van Loan [22].

Symmetric positive definiteness. Let $A \in \mathbb{R}^{n\times n}$. A is said to be symmetric if $A = A^T$. If A is symmetric and if

$$x^T Ax > 0$$

for all nonzero $x \in \mathbb{R}^n$, then A is said to be symmetric positive definite.

QR factorization. The QR factorization of $A \in \mathbb{R}^{m\times n}$ is a decomposition

$$A = QR,$$

where $Q \in \mathbb{R}^{m\times n}$ has orthonormal columns and $R \in \mathbb{R}^{n\times n}$ is upper triangular. The matrix Q forms an orthonormal basis for range(A).

LU decomposition. The LU decomposition of $A \in \mathbb{R}^{n\times n}$ is a factorization

$$A = LU,$$

where L is a lower triangular matrix and U an upper triangular matrix. A has an LU decomposition under the condition that $\det(A[1:k, 1:k]) \neq 0$ for $k = 1, \ldots, n-1$. More generally, any square matrix A has an LU decomposition with partial pivoting (LUP), $PA = LU$. Computing the LU decomposition costs $\mathcal{O}(n^3)$ flops, specifically, $\frac{2}{3}n^3$.

Cholesky factorization. Assume that $A \in \mathbb{R}^{n\times n}$ is SPD. Then A has a unique Cholesky factorization

$$A = LL^T,$$

where L is a lower triangular matrix with positive diagonal elements. This is a special case of the LU decomposition, with $U = L^T$. The cost of this procedure is $\mathcal{O}(n^3)$ flops, specifically, $\frac{1}{3}n^3$, making it faster than standard LU decomposition.

Incomplete Cholesky factorization. If $A \in \mathbb{R}^{n\times n}$ is SPD with Cholesky factorization $A = LL^T$, then the incomplete Cholesky factorization is an approximation

$$A \approx \tilde{L}\tilde{L}^T,$$

where $\tilde{L}$ is a lower triangular matrix that is sparser than L. There are many strategies for computing the incomplete Cholesky, see Lin and Moré [53]. The main advantage over the full Cholesky factorization is reduced storage and computational cost, making it particularly effective for large sparse SPD matrices.

A.4 Solving triangular systems

Let $R \in \mathbb{R}^{n\times n}$ be triangular. Solving $Rx = y$ is done by forward (or backward) substitution: starting from the first (or last) row, each x_i is solved directly after substituting all previously computed values. If R is sparse, zero entries are skipped, and by Lemma 2.2.1 in George and Liu [31] the cost reduces to $\mathcal{O}(\text{nnz}(R))$, where nnz($R$) is the number of nonzeros in R.

General linear systems. Let $A \in \mathbb{R}^{n\times n}$ be invertible and consider $Ax = y$, then the solution is $x = A^{-1}y$. In practice, the standard procedure is to compute the LU factorization $A = LU$, reducing the problem to two triangular systems

$$Lz = y, \quad Ux = z \implies x = U^{-1}z = U^{-1}L^{-1}y.$$

Each triangular system can be solved by forward and backward substitution at $\mathcal{O}(n^2)$ [22]. The factorization itself costs $\frac{2}{3}n^3$ flops, so the problem scales as $\mathcal{O}(n^3)$. If A is symmetric positive definite, $U = L^T$ and the Cholesky factorization $A = LL^T$ halves the cost to $\frac{1}{3}n^3$ (Appendix A.3).

Sparse linear systems. For sparse systems $Ax = y$, computing the LU factorization comes with additional challenges. Matrix entries that are zero in A but non-zero in L or U, so-called fill-ins, require more memory and result in a more expensive factorization [18]. Additionally, the cost of solving the resulting triangular systems is higher. Much research has gone into the development of algorithms which avoid fill-ins, see Davis [54] for an overview.

The nested dissection method. For sparse SPD matrices of size $N \times N$ arising from 2D elliptic PDEs (mass and stiffness matrices), the structure of the matrices can be exploited. Alan George proposed the nested dissection method in 1973 [30], a divide and conquer approach that computes Cholesky factorization in $\mathcal{O}(N^{3/2})$ flops. The resulting triangular matrix L contains $\mathcal{O}(N \log N)$ non-zero elements, so solving $Lx = y$ costs $\mathcal{O}(\text{nnz}(L)) = \mathcal{O}(N \log N)$.

A.5 Dynamical low-rank approximation

In [35], Koch and Lubich study the problem of finding a rank-r approximation $X(t)$ to a time-dependent data matrix $A(t)$,

$$\min_{X(t)\in\mathcal{M}_r} \|X(t) - A(t)\|_F,$$

where $\mathcal{M}_r = \{M \in \mathbb{R}^{m\times n} : \text{rank}(M) = r\}$. Instead of computing a best approximation via SVD at every time t, they propose a dynamical approach. They seek a low-rank matrix $Y(t) \in \mathcal{M}_r$ such that its derivative $\dot{Y}(t)$ approximates $\dot{A}(t)$:

$$\min_{\dot{Y}(t)\in\mathcal{T}_{Y(t)}(\mathcal{M}_r)} \|\dot{Y}(t) - \dot{A}(t)\|_F, \tag{A5}$$

where $\mathcal{T}_{Y(t)}(\mathcal{M}_r)$ denotes the tangent space of $\mathcal{M}_r$ at $Y(t)$.

Let $Y = U\Sigma V^T$ be a factorization where $U \in \mathbb{R}^{m\times r}$ and $V \in \mathbb{R}^{n\times r}$ have orthonormal columns, and $\Sigma \in \mathbb{R}^{r\times r}$ is non-singular (using the SVD yields a diagonal Σ, but this is not a necessary assumption). Koch and Lubich show that Equation (A5) is equivalent to the Galerkin condition: find $\dot{Y} \in \mathcal{T}_Y(\mathcal{M}_r)$ such that

$$\langle \dot{Y} - \dot{A},\, \delta Y\rangle_F = 0, \quad \text{for all } \delta Y \in \mathcal{T}_Y(\mathcal{M}_r).$$

By defining orthogonal projectors $P_U^\perp = I - UU^T$ and $P_V^\perp = I - VV^T$ they prove (Proposition 2.1) that this condition yields the following system of matrix differential equations:

$$\begin{aligned}\dot{\Sigma} &= U^T \dot{A} V,\\ \dot{U} &= P_U^\perp \dot{A} V \Sigma^{-1},\\ \dot{V} &= P_V^\perp \dot{A}^T U \Sigma^{-T}.\end{aligned}$$

These equations allow for evolution of the low-rank factors using numerical integration techniques.

A.6 Conjugate gradient algorithm

Consider the quadratic function $f(x) = \frac{1}{2}x^T Ax - y^T x$ where A is SPD, which has the gradient $\nabla f(x) = Ax - y$. By the optimality condition $\nabla f(x) = 0$, the minimum of the function is the solution of $Ax = y$.

Consider the update rule $x_{k+1} = x_k + \alpha_k d_k$, where α_k is a step-size and d_k some search direction. The optimal α_k is the minimizer of the function $h(\alpha) = f(x_k + \alpha d_k)$, and setting $h'(\alpha) = 0$ with $d_k = -\nabla f(x_k)$ yields the step-size

$$\alpha_k = \frac{r_k^T r_k}{d_k^T A d_k}, \quad r_k = y - Ax_k.$$

This is the steepest descent method [13]. However, if the condition number of A is high, then the contours of f get stretched out, giving an elliptic landscape. Using the steepest descent rule, we have that $d_{k+1}^T d_k = 0$ [13], so the search directions will zig-zag across the landscape.

Instead, if we seek search directions that are A-orthogonal, $d_{k+1}^T A d_k = 0$, we are moving within a transformed landscape where the contours of f are perfectly circular. Choosing d_{k+1} as a linear combination of $-\nabla f(x_{k+1}) = r_{k+1}$ and the previous direction d_k,

$$d_{k+1} = r_{k+1} + \beta_k d_k,$$

enforcing $d_{k+1}^T A d_k = 0$,

$$(r_{k+1} + \beta_k d_k)^T A d_k = 0,$$

and solving for β_k, leads to

$$\beta_k = -\frac{r_{k+1}^T A d_k}{d_k^T A d_k}.$$

This is the conjugate gradient algorithm [13].

The A-term in the formula of β_k can be computationally expensive to compute if f is non-linear [13]. The formula of β_k can be rewritten without A to obtain the Fletcher-Reeves formula [16],

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}.$$

Alternatively, the Hestenes-Stiefel formula [15] computes β_k as

$$\beta_k = -\frac{r_{k+1}^T(r_{k+1} - r_k)}{d_k^T(r_{k+1} - r_k)}.$$

Another well-known modification is the Polak-Ribière formula [55],

$$\beta_k = \frac{r_{k+1}^T(r_{k+1} - r_k)}{r_k^T r_k}.$$

A.7 Performance metrics

Let $x \in \mathbb{R}^N$ denote the true source (its coefficient vector) and $\hat{x} \in \mathbb{R}^N$ its corresponding reconstruction. To evaluate reconstruction quality, we consider three performance metrics: Euclidean error, EMD, and AUC-IoU.

Earth mover's distance. The earth mover's distance measures the dissimilarity between two distributions, which in our case refers to x and $\hat{x}$. Let x_d and $\hat{x}_d$ be the normalized distributions of x and $\hat{x}$, respectively, computed as

$$(x_d)_i = \frac{|x_i|}{\sum_{j=1}^N |x_j|}, \quad (\hat{x}_d)_i = \frac{|\hat{x}_i|}{\sum_{j=1}^N |\hat{x}_j|}.$$

Let $p_i \in \mathbb{R}^2$ denote the spatial coordinates of the i-th node in the mesh. The EMD between x_d and $\hat{x}_d$ is

$$\mathrm{EMD}(x_d, \hat{x}_d) = \inf_{\gamma \in \Pi} \sum_{i,j=1}^{N} \gamma_{ij} \, \|p_i - p_j\|,$$

where Π is the set of all joint distributions with marginals x_d and $\hat{x}_d$ [56].

AUC-IoU. The IoU, known as the Jaccard index in statistics, measures the similarity between two sets. It is defined as the size of intersection divided by the size of the union [57].

We define the binary mask of x at threshold $\tau \in (0, 1)$ as the set

$$S_\tau(x) = \{i : x_i \geq \tau \max(x)\}.$$

Then, the IoU of x and $\hat{x}$ at threshold τ is the IoU of their binary masks,

$$\mathrm{IoU}(\tau) = \frac{|S_\tau(x) \cap S_\tau(\hat{x})|}{|S_\tau(x) \cup S_\tau(\hat{x})|}.$$

Here, $|\cdot|$ denotes the number of elements of a set (its cardinality).

The AUC-IoU is then the integration of the $\mathrm{IoU}(\tau)$ curve,

$$\text{AUC-IoU} = \int_0^1 \mathrm{IoU}(\tau) \, d\tau.$$

This can be approximated using the Trapezoidal rule. The AUC-IoU lies in $[0, 1]$, with 1 being a perfect score.

Norges miljø- og biovitenskapelige universitet
Noregs miljø- og biovitskaplege universitet
Norwegian University of Life Sciences

Postboks 5003
NO-1432 Ås
Norway